

RESEARCH ARTICLE

Deep Learning-Based Drowsiness Detection System for Driver's Safety

SINDHU VIDYANATHAN DIXITH¹, (Member, IEEE),
SHRIKANT JADHAV¹, (Senior Member, IEEE), YOUNGSOO KIM²,
AND NAVEENKUMAR JAYAKUMAR³

¹Department of Electrical Engineering, San José State University, San Jose, CA 95192, USA

²Department of Electrical Engineering, University of Minnesota Duluth, Duluth, MN 55812, USA

³School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India

Corresponding author: Shrikant Jadhav (shrikant.jadhav@sjsu.edu)

ABSTRACT Driver drowsiness is a leading contributor to road accidents, accounting for over 100,000 crashes and approximately ~1000 fatalities each year in the United States alone, as per National Safety Council (NSC). To mitigate this urgent public-safety risk, we propose a real-time Driver Drowsiness Detection System that achieves both high accuracy and fast inference on standard hardware. Our key idea is to combine two complementary deep-learning strategies: 1) a custom Convolutional Neural Network (CNN) paired with a Support Vector Machine (SVM) classifier, and 2) a lightweight transfer-learning model built upon a pre-trained convolutional backbone. We evaluate these approaches on two datasets: a four-class Kaggle collection of open/closed eyes and yawn/no-yawn images, and the 37-subject MRL eye dataset. For the custom CNN+SVM pipeline, we optimized the split ratios, dropout rates, and L2 regularization to achieve 100% training accuracy and 99.7% validation accuracy. For the transfer-learning model, we leveraged an existing network to accelerate training, achieving 99.4% training accuracy and 99.1% validation accuracy. Finally, we compare both models using metrics including loss curves, confusion matrices, precision, recall, and F1-score. Our system demonstrates that real-time, highly accurate drowsiness detection is achievable without specialized hardware, paving the way for broader deployment in road-safety applications.

INDEX TERMS Convolutional neural network (CNN), driver drowsiness, deep learning, eye aspect ratio (EAR), L2 regularization, mouth aspect ratio (MAR), OpenCV, support vector machine (SVM), transfer learning.

I. INTRODUCTION

Driver drowsiness is a leading cause of road accidents, posing a significant threat to global road safety. According to data from the National Highway Traffic Safety Administration (NHTSA) [1], thousands of crashes and hundreds of fatalities occur each year due to drivers falling asleep or losing focus at the wheel. This alarming trend highlights the urgent need for practical solutions to detect and prevent drowsy driving before it leads to catastrophic consequences. Beyond impaired reaction times and lapses in attention, drowsiness often manifests subtly—through prolonged eye closures or yawning—that go unnoticed until it is too late. This urgent public-safety challenge demands a real-time detection system

capable of alerting drivers before fatigue culminates in a catastrophic accident.

Traditional drowsiness detection has predominantly relied on intrusive wearables (e.g., EEG headbands, smartwatches) or cumbersome sensors [2], [3], [4]. Although these methods can achieve high accuracy under controlled conditions, they suffer from limited user acceptance, inconsistent usage, and poor adaptability to varying environments. Computer-vision approaches using Eye Aspect Ratio (EAR) or Mouth Aspect Ratio (MAR) have shown promise [5], but most systems focus on a single cue (eye closure or yawning) and often lack the real-time speed or robustness needed for field deployment.

To better understand the scale of the problem, we conducted a comprehensive analysis of data from the NHTSA, which revealed that over 800 fatalities and 700 crashes

The associate editor coordinating the review of this manuscript and approving it for publication was Aysegül Ucar¹.

occur annually due to drowsy driving. Our study involved examining the circumstances of each crash, including the time of day, road conditions, and driver behavior, to identify patterns and trends. Figure 1 illustrates this trend, showing the number of crashes and deaths between 2017 and 2021. As shown in the figure, the number of deaths due to drowsy driving has been constantly increasing since 2018, and 29834 deaths have occurred from 2017 to 2021. These numbers underscore the pressing need for innovative solutions to address this pervasive issue.

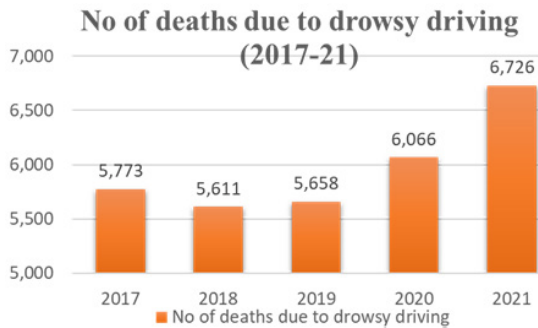


FIGURE 1. NHTSA traffic safety crashes and fatalities [1].

To overcome these limitations, we propose a non-intrusive, webcam-based Driver Drowsiness Detection System that fuses two complementary strategies: (1) **Custom CNN + SVM Pipeline**: We extract **EAR and MAR** features from facial landmarks and then feed them into a **lightweight Convolutional Neural Network (CNN)**, whose **bottleneck features are classified using a Support Vector Machine (SVM)**. We optimize dropout, L2 regularization, and train/validation splits to maximize accuracy. (2) **Transfer Learning Model**: We fine-tune a pre-trained convolutional backbone on our drowsiness datasets, dramatically reducing training time while maintaining high performance.

The primary goal of this research is to develop a robust, accurate, and practical solution that alerts drivers when they exhibit signs of drowsiness, thereby preventing accidents and saving lives. By implementing our Driver Drowsiness Detection System, we believe we can significantly reduce the number of drowsy-driving-related crashes and fatalities, making our roads safer for everyone. Moreover, the system's potential applications in industries involving heavy machinery or hazardous materials could revolutionize safety standards in these high-risk settings. Through this work, we aim to contribute to the growing body of knowledge in driver safety systems and demonstrate the potential of combining traditional computer vision techniques with modern deep learning architectures to tackle real-world problems.

The novelty of our study lies in its innovative combination of hybrid EAR and MAR algorithms with advanced machine learning models, including CNN, SVM, and Transfer Learning. Unlike traditional approaches that rely solely on eye closure or yawning detection, our hybrid algorithm provides

a more comprehensive understanding of drowsiness by analyzing both facial features simultaneously. Furthermore, we enhance CNN models using data augmentation, L2 regularization, dropout, and varying training-validation split ratios, achieving near-perfect accuracy. Transfer learning significantly reduces training time while maintaining high performance, making our system practical for real-time applications. This integrated approach improves accuracy and ensures robustness across diverse scenarios, addressing the limitations of existing systems.

II. RELATED WORK

Several studies have been published on drowsiness detection. Some have used various algorithms, whereas few have used deep learning methods for driver drowsiness detection. Drowsiness detection can be modeled using these models. A mathematical model was developed using statistics and data analysis [6]. The sleep duration of the participants was analyzed and modeled. Another method uses fuzzy logic, in which fuzzy rules and functions are used for analysis. Machine learning methods include model development using Support Vector Machines (SVMs) or deep learning models, such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) [7], [8], [9]. Alshaqai focused on an algorithm for drowsiness detection using face detection and eye tracking [10], [11]. Eye state and PERCLOS were measured using the HAAR algorithm and OpenCV for detection. Razaee mentioned that PERCLOS features the creation of a real-time intelligent alarm system for eye status detection that takes video sequences and determines whether a driver is drowsy. Eyelid movements were used to calculate human-defined eye features. This provides information on eyelid movements. The utilization of these artificial features provides additional information for model development [12], [13], [14].

Yong discussed the identification of drowsiness through emotion recognition, such as smile detection. If no emotion was detected, the drowsiness condition was detected. This algorithm is complex and requires time to train [15], [16]. Choi et al. discussed categorization that can be achieved by developing deep learning models for facial recognition using driver gaze zones. This model implementation requires considerable data and works if a high accuracy is obtained [17], [18]. Trivedi considered only iris status for drowsiness detection. This study focuses on head analysis, eye movement, and iris movement. Head movement is not considered in the development, which could yield better results [19], [20].

In [21], [22], and [23], the MRL dataset was used to leverage transfer learning for vehicle safety. An accuracy of 97% was obtained for the drowsiness detection. Assari and Rahmati used facial expressions to detect drowsiness in their study. They used a hardware system to detect the face using infrared light to detect drowsiness [24], [25], [26]. The eye aspect algorithm and the Raspberry Pi 4 were used to detect

drowsiness. It performed well under light conditions, with an accuracy of 90% [27]. MOR values are used to detect the state of drowsiness [28].

The authors used the HOG SVM approach to detect drowsiness and compared it with that of a human rater. The popularity of the HOG SVM lies in object detection and not eye blink detection. Furthermore, comparisons with a human rater are highly unreliable because human decisions are prone to more errors than automated ones [3], [29]. This study proposes a smartwatch and headband containing sensors for identifying drowsiness. However, this is limited by the choice of the driver, who may prefer to avoid wearing a headband or a different watch [2]. Singh et al. developed a reliable drowsiness monitor that integrates an electrode structure within a steering wheel. The device measures the heart rate using non-contact capacitive electrocardiograph (ECG) signals. This approach is still in its nascent phase [30]. Quddus, Shahidi Zandi, and Prest used the LSTM to detect drowsiness. Several models were developed, and an accuracy of 82% was obtained using R-LSTM. Using C-LSTM, they obtained an accuracy of 95-97%. They used a highly complex algorithm to train their models. However, its advantage is that it considers head movements [31], [32].

Another study using the CNN-LSTM model was developed for ECG classification. The developed model had five classes and an accuracy of 99.26% was obtained [33]. The authors used transfer learning with the AlexNet model to train a pre-trained convolutional neural network to classify four classes. An accuracy of 97.89% was obtained [34]. [35] focused on the discrete wavelet transform, and the developed model had five classes. An accuracy of 98.80% was achieved. Using a CNN for cardiac detection, an accuracy of 97.42% was obtained for five classes [36]. A convolutional neural network was used for myocardial infarction to apply ECG signals. An accuracy of 93.53% was obtained [37]. An accuracy of 98.18% was obtained using multi-scale convolutional networks [38], [39], [40]. Fibrillation detection was implemented to obtain the heart rate with an accuracy of 97.83% [41].

Much research has been done in this area. We briefly summarize existing literature on driver drowsiness detection in Table 1. In the table, we illustrated the dataset used for the training and classification methods used and described the algorithms used to detect drivers' drowsiness. Taxonomy and definitions of terminologies are presented in review papers [42], [43]. Our discussion focuses on how drowsiness detection systems are implemented, especially deep learning. In addition to these approaches, various ad hoc methods and techniques exist for estimating eye state in a drowsiness detection system design and optimizing CNN subsystems. The authors of [44] build a CNN model for eye state categorization using an MRL eye data set. The 4D CNN model has a convolution layer followed by an activation function, batch normalization, and ReLU step. The accuracy of the 4D model was tested and compared with pre-trained

models, including VGG16 and VGG19, which are widely used in computer vision tasks. The 4D model performed very well in terms of 97.53% accuracy.

Our survey of existing drowsiness-detection methods reveals three dominant trends: (1) single-cue approaches (e.g. PERCLOS or yawning detection) that achieve moderate accuracy but miss complementary fatigue indicators; (2) intrusive sensor-based systems (e.g. wearable EEG or ECG) that offer high reliability under controlled conditions but suffer from poor user adoption and environmental constraints; and (3) deep-learning frameworks that deliver state-of-the-art accuracy yet often require extensive labeled data and specialized hardware for real-time inference. Few studies have explored hybrid, multi-cue algorithms that jointly analyze eye and mouth features, and even fewer have demonstrated real-time performance on standard webcams and CPUs. This gap motivates our work: we propose a non-intrusive, hybrid EAR+MAR detection pipeline, implemented both as a custom CNN+SVM and a transfer-learning model, and benchmarked for real-time feasibility on commodity hardware—thereby addressing both accuracy and deployment challenges left open in prior literature.

A. OUR CONTRIBUTIONS

Through this work, we aim to contribute to the expanding knowledge base in driver safety systems and demonstrate the potential of combining traditional computer vision techniques with modern deep learning architectures to tackle real-world problems. We focused on developing a robust method for detecting drowsiness using hybrid algorithms, using EAR and MAR to analyze and detect drowsiness in real-time. The main contributions of this work are summarized as follows.

- Our work introduces a novel hybrid algorithm that combines EAR and MAR, a unique approach that will pique your interest. This complete implementation can perform end-to-end drowsiness detection without human intervention, a feature that significantly reduces incorrect predictions and holds great potential for the field.
- Transfer learning informed CNN and SVM: We experimented with transfer learning, a technique that involves using pre-trained models as a starting point for training a new model, thereby reducing the amount of data and time required for training. This approach has proven to be a promising technique for image recognition and eye state predictions in drowsiness detection. Very few efforts have been made to date to combine transfer learning and deep neural learning, including CNN and SVM, in real-world settings with real-world MRL data.
- Through rigorous and comprehensive evaluations, we have thoroughly studied the performance and accuracy of our approaches. Our models achieve high accuracy, and we perform a comparative analysis with

TABLE 1. Summarized literature.

Reference	Dataset	Classification Methods	Descriptions
[9], [10]	Real-time images from the camera setup	Eye state and PERCLOS are measured using the HAAR classifier	These papers focus on the algorithm for drowsiness detection using face detection and eye tracking.
[14]	NTHU, OffSec Datasets	Local detection CNN model combined with global detection of car path using YoLov5	Authors are identifying drowsiness through emotion recognition.
[16]	Custom video dataset of 7200 frames	Face tracking with a single camera and a convolutional neural network (CNN) model	Facial recognition using drivers' gaze zones.
[20]–[22]	Motion sensor data collected at the wrist	HOG and SVM	They proposes a smart watch and a headband containing sensors to identify drowsiness.
[32]	ECG Dataset	Used AlexNet model to train a pre-trained convolutional neural network	A convolutional neural network is used for myocardial infarction to apply ECG data.

previously published results, instilling confidence in the robustness of our findings.

- Our work provides a deployable solution with live alerts and real-world usability, offering a promising outlook for the practical application of our research in enhancing driver safety.

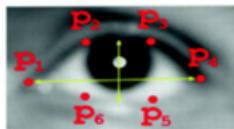
The article is organized as follows. Section III explains the EAR, MAR, and hybrid EAR and MAR algorithms were used to implement the drowsiness detection system. This section also describes the machine learning models developed using CNN and SVN, transfer learning, and model predictions. Section IV discusses the experimental results obtained using individual and hybrid EAR and MAR implementations. The section also presents the results obtained using ML models, CNN and SVM, and the transfer learning methodology. Section VI concludes the paper.

III. DROWSINESS DETECTION SYSTEM

A. EAR, MAR, AND HYBRID ALGORITHM IMPLEMENTATION

1) EAR ALGORITHM

We used EAR to process the eye features and analyze drowsiness detection. The camera captured a live stream of the driver's facial image. In our approach, the driver does not need to wear any additional gadget or sensors on the body. The physical signs of the facial images were analyzed using a drowsiness detection algorithm to ensure reliability and accuracy. An EAR threshold value was set to determine drowsiness. If the threshold value is satisfied, the person can be notified by raising an alarm.



$$EAR = \frac{\|P_2 - P_6\| + \|P_3 - P_5\|}{2 \times \|P_1 - P_4\|} \quad (1)$$

The upper and lower eye landmarks P_2 , P_6 , P_3 , and P_5 , and the edge landmarks P_1 and P_4 are shown in Equation 1. In our study, we bifurcated the left and right eye. According to this equation, the ratio was obtained as the sum of the

distances between P_2 and P_6 , P_3 and P_5 divided by the distance between P_1 and P_4 . Based on age, gender, and race, landmark values change for every person.

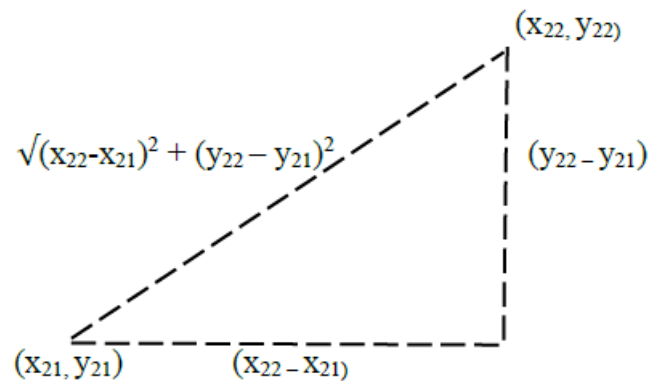


FIGURE 2. Euclidean distance between two points.

We used the Euclidean distance formula to calculate the distance between the two points and computed the EAR value, as shown in Figure 2. The Euclidean distance is the square root of the sum of the squared distances between the X- and Y-coordinates. Figure 3 shows the landmarks of the overall facial image. OpenCV was used to identify these landmarks in live facial video images streamed using a webcam. *Sleep*, *Drowsy*, and *Active* states were defined to distinguish the different states for drowsiness detection. The EAR value is calculated by computing the distance between two landmark points. We read landmarks [42], [43], [44], [47], [46], and [45] for the right eye. For the left eye, landmarks [36], [37], [38], [41], [40], and [39] were read.

Figure 4 shows a flow chart of the EAR algorithm used to calculate EAR values. The camera, face, and landmark detectors were initialized. In real-time eye blinking, landmarks are captured to compute EAR using the Euclidean distance formula. The drowsiness detection states, that is, *sleepy*, *drowsy*, and *active*, were defined based on the computed EAR values of the live image streaming. Each state was assigned a counter based on the eye state. An individual is in an *Active* state if the EAR value is greater than the 0.25 threshold. If the value is between 0.21 and 0.25, the



FIGURE 3. Facial landmarks.

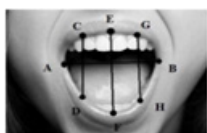
person is in a *Drowsy* state. If the counter value exceeds six and the EAR threshold value is below 0.21, the person is in a *Sleep* state.

Real-time data analysis was performed to determine the threshold value based on the eye status. In safety applications, such as drowsy driving, the need for a stricter threshold, as in the case of driver safety, is of utmost importance. A larger dataset of images from various individuals was analyzed by studying the facial features. Fine-tuning may be required, depending on the characteristics of the individual being monitored.

Algorithm 1 shows the pseudo-code for the EAR detection. All counters, that is, sleep, drowsy, and active, were set to zero. The *compute* function was used to calculate the distance between two points using the Euclidean method. The right- and left-eye landmarks were passed as parameters for the *blinked* function to calculate the ratio value using the EAR formula. If the ratio is greater than 0.25, then function returns 2. The function returns 1 and 0 if the ratio is between 0.21 to 0.25 and below 0.21, respectively. When one or both eyes return 0 and the sleep counter is incremented beyond 6, the status is set as sleeping. The status is set to drowsy if both eyes return 1 and the drowsy counter exceeds 6. Otherwise, the status will be set to active.

2) MAR ALGORITHM

The implementation of the MAR ratio, which is a key element in determining a person's yawning state, was facilitated using the Dlib library. This library was instrumental in providing frontal face detectors and landmark predictor [21], which were essential for our implementation. The MAR is the ratio of the distance between the upper and lower lips to the distance between the side lips, as shown in Equation 2. OpenCV was used to identify the lip landmarks.



$$MAR = \frac{|EF|}{|AB|} \quad (2)$$

The implementation flow-chart in Figure 5 illustrates the steps involved in yawning detection from live feed

Algorithm 1 EAR Detection

Require: Facial landmarks, sleep = 0, drowsy = 0, active = 0, status = ' '

Ensure: To check the EAR ratio and detect drowsiness

```

1: function Compute(a, b)
2:   return Euclidean distance between points a and b
3: end function
4: function Blinked(a, b, c, d, e, f)
5:   up ← Compute(b, d) + Compute(c, e)
6:   down ← Compute(a, f)
7:   ratio ← up / (2.0 * down)
8:   if ratio > 0.25 then
9:     blinked ← 2
10:  else if 0.21 < ratio ≤ 0.25 then
11:    blinked ← 1
12:  else
13:    blinked ← 0
14:  end if
15:  return blinked
16: end function
17: right_blink ← Blinked(landmarks[42], landmarks[43],
    landmarks[44], landmarks[47], landmarks[46], landmarks[45])
18: left_blink ← Blinked(landmarks[36], landmarks[37],
    landmarks[38], landmarks[41], landmarks[40], landmarks[39])
19: for face on live detection do
20:   if left_blink == 0 or right_blink == 0 then
21:     if sleep > 6 then ▷ The counters are cleared
22:       status ← "Sleeping"
23:   end if
24:   else if left_blink == 1 or right_blink == 1 then
25:     if drowsy > 6 then ▷ The counters are cleared
26:       status ← "Drowsy"
27:   end if
28:   else
29:     if active > 6 then ▷ The counters are cleared
30:       status ← "Active"
31:   end if
32:   end if
33: end for

```

received from the camera. The landmarks were identified and initialized, and individual functions were implemented to detect the top and bottom lips using these landmarks. These functions, which detect the face and calculate the MAR ratio, primarily focus on lip distance. Lip distance was computed by subtracting the absolute value from the top to the bottom lip. Live detection was initiated using a web camera. If the lip distance exceeds a particular threshold value, an alarm alerting the driver is triggered. The yawn count was stored on the counter and the number of times the driver had yawned was recorded.

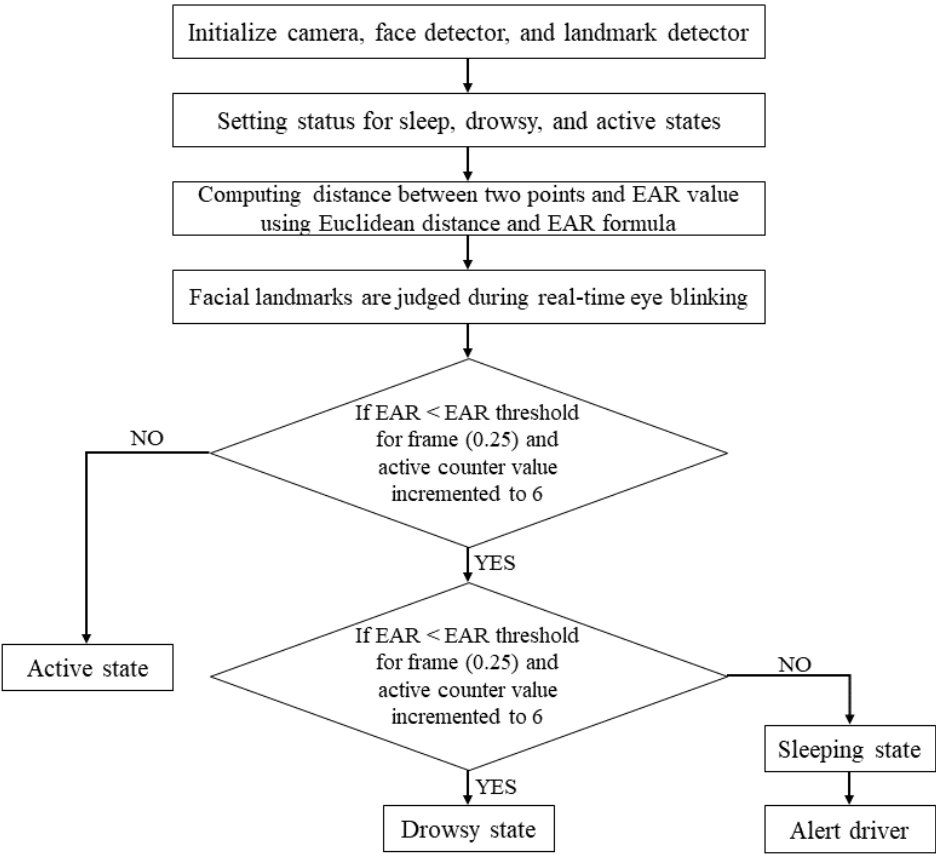


FIGURE 4. EAR algorithm flow chart.

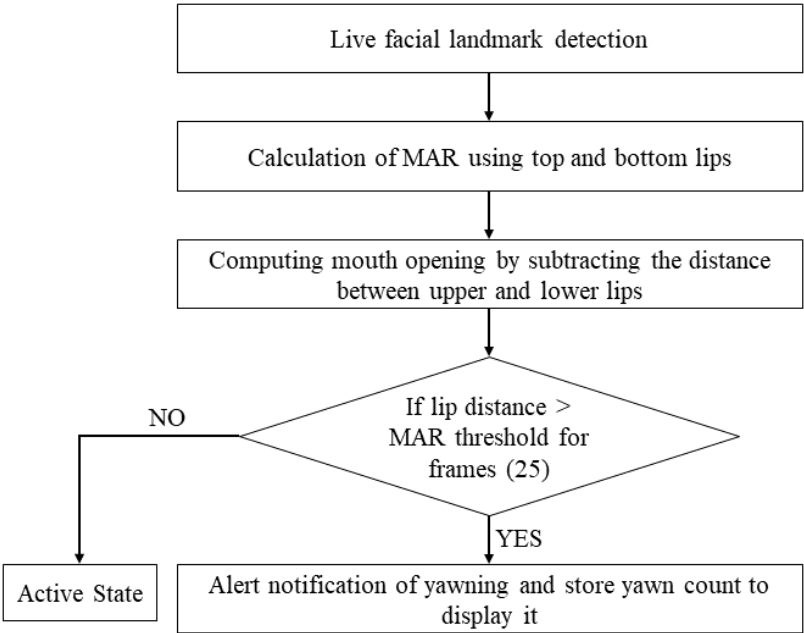


FIGURE 5. MAR algorithm flow chart.

Algorithm 2 presents the pseudo-code for computing MAR. The center of the top lip was used to calculate the

landmarks of the upper lip. The mean value was calculated for the top lip by using these landmarks. Similarly, the

mean value of the lower lip was obtained. Lip distance was calculated by subtracting the absolute values of the top and bottom lip centers. In live detection, when the lip distance exceeds the threshold value, yawn_status is set to True, and “Yawning, be alert” is displayed. Previous yawn counts were stored and displayed.

Algorithm 2 MAR Detection

Require: landmarks, yawns = 0, yawn_status = False

Ensure: To check the MAR lip distance and detect drowsiness

```

1: landmarks  $\leftarrow$  get_landmarks(image)
2: function mouth_open(image)
3:   top_lip_center  $\leftarrow$  top_lip(landmarks)
4:   bottom_lip_center  $\leftarrow$  bottom_lip(landmarks)
5:   lip_distance  $\leftarrow$  |top_lip_center - bottom_lip_center|
6:   return lip_distance
7: end function
8: for each face in live detection do
9:   lip_distance  $\leftarrow$  mouth_open(image)
10:  if lip_distance > 25 then
11:    yawn_status  $\leftarrow$  True
12:    Display Message: “Yawning, Be alert”
13:  else
14:    yawn_status  $\leftarrow$  False
15:  end if
16:  if prev_yawn_status == True and yawn_status == False then
17:    yawns + = 1
18:  end if
19: end for
  
```

3) HYBRID ALGORITHM

Figure 6 presents an overview of the hybrid system. The driver's facial images are captured via camera. We extracted the features using facial landmarks and region of interest selection. These features were then used for a comprehensive training process involving techniques such as basic CNN models, data augmentation, L2 regularization, complex neural networks, dropout results, and varying training and validation datasets. The models trained using our techniques were utilized as inferences to classify drivers' status and make decisions.

We innovatively combined EAR and MAR algorithms to implement a drowsiness detection system that offers unique benefits. In real-time, our hybrid algorithm identifies the eyes and mouth, thereby providing a more comprehensive understanding of drowsiness. This novel approach reduces likelihood of incorrect predictions, enhancing the system's overall performance. This added functionality is a significant advantage when it is challenging to determine drowsiness based solely on the eyes. The pseudo-codes mentioned in Algorithm 1 and 2 were seamlessly integrated.

Figure 7 illustrates the practical application of the proposed hybrid algorithm. The EAR function was implemented using

the Euclidean distance between the two sets of eyes. We used cv2 for live-landmark detection. The lip distance, calculated from the top lip center to the bottom lip center, was used to determine if a yawn occurred. We set a threshold value of 25 for lip distance, considering it a yawn if it exceeded this value. Yawn counts were recorded using a counter. An alarm was triggered if the eyes were closed to a certain threshold, indicating drowsiness.

The combination of the EAR and MAR algorithms significantly enhanced the system's reliability and proved to be a superior approach compared with implementing them separately. Prediction accuracy was notably high, instilling a sense of trust in the system's capabilities. Detailed comparisons are presented in Section IV, which further highlights the advantages of the proposed hybrid algorithm.

B. CONVOLUTIONAL NEURAL NETWORK (CNN) AND SUPPORT VECTOR MACHINE (SVM) MODELS FOR DROWSINESS DETECTION

For our application, we developed two machine-learning models. In one method, we used deep learning methods, CNN and SVM, and in another method, we developed a transfer learning model for drowsiness detection. OpenCV was used to split the eyes into left and right for eye detection. The model predicted both open and closed eyes, with a negative value for closed eyes and positive value for open eyes. A timer was used to detect the time when eyes were closed. If the timer is set for a particular threshold value, a drowsy state is asserted and an alarm is raised to alert the driver.

Two datasets were used to train, test, and validate the proposed model, respectively. The first dataset was a four-label dataset containing open-eye, closed-eye, yawning, and no-yawning images. We used a dataset from Kaggle. There were two folders: the training and test datasets. The test and training datasets contain open-eye, closed-eye, open-mouth, and closed-mouth images. The training dataset contained 2467 images, and the test dataset contained 433 images. The other dataset is an open-source MRL eye dataset. It included images of 37 individuals (33 men and four women). It was not intended for drowsiness but rather only for pupil detection. This dataset was used to detect drowsiness. The eyes were divided into closed and open eyes, and grouped separately.

1) DROWSINESS DETECTION SYSTEM USING CNN MODEL

The CNN, a deep learning model, is extensively used in image processing and computer vision applications. It is particularly effective in handling grid-type datasets such as time-series data (signals), image data, and multidimensional data. We assigned weights to each pixel in the CNN by incorporating parameter sharing and sparse interactions. The kernel, which is smaller than the input, uses sparse weight. The pooling layer replaces the output with summary statistics using two types of pooling: max and average. Max pooling selects the maximum value within neighborhood cells, whereas average pooling selects the weighted average

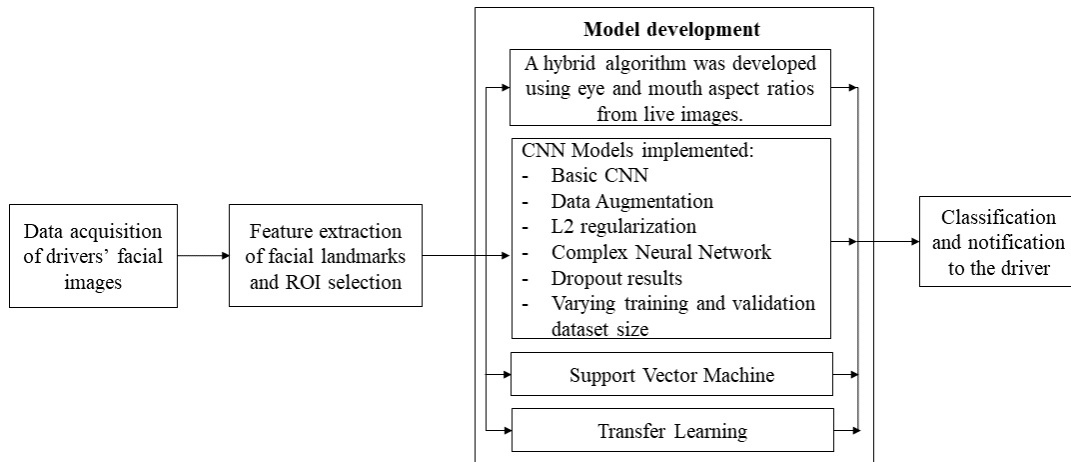


FIGURE 6. System overview.

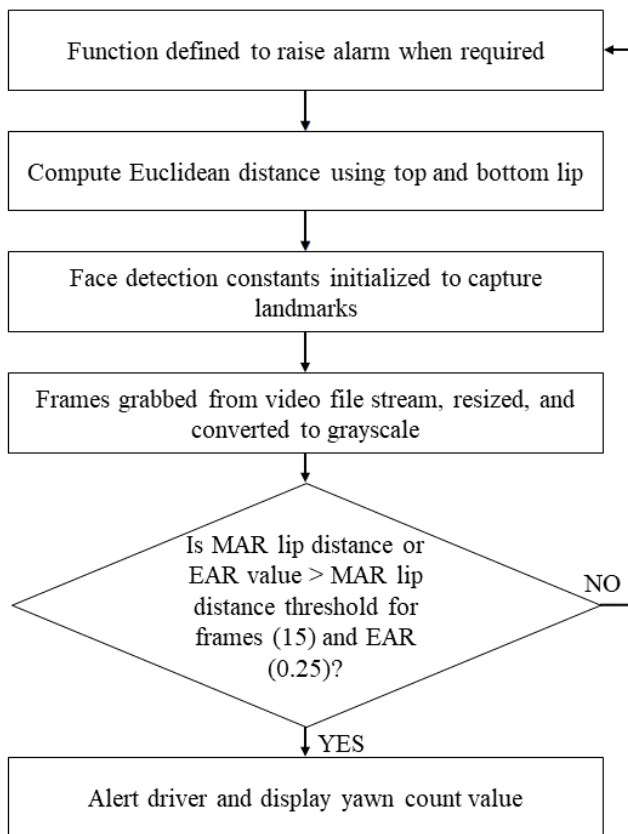


FIGURE 7. Hybrid algorithm flow chart.

of the neighborhood cells. Stride, defined as the number of shifts during convolution and pooling, can yield a smaller or larger grid. Convolutional and pooling layers comprise multiple feature maps and kernel sizes. The kernel's effects on an image can range from no effect to an averaging effect, shift left by several pixels, or edge detection. The padding technique ensures the same size for input and output images.

A fully connected or dense layer with an activation function is defined at the end of a CNN model.

The CNN model, developed using the MRL and the four-label dataset, demonstrated its precision in drowsiness detection. The images were pre-processed and converted into grayscale images. Figure 8 shows the CNN architecture, which includes three convolutional layers with 32 and 64 feature maps, 3×3 kernel size, and Rectified Linear Unit (ReLU) activation function. It also incorporates three max-pooling layers with 2×2 kernel size, followed by a dense layer with ReLU and softmax activation functions, further enhancing its accuracy.

As previously mentioned, we used a four-label dataset to train our model. The shape of the training image was reshaped to an image size of 48×48 . A class map was created for each of the four labels: 0 for yawn, 1 for eyes open, 2 for no yawn, and 3 for eyes closed. The images are displayed in a grid format, as shown in Figure 9. In total, 2900 available images were available. A total of 2030 (70%) images were used for training, 435 (15%) images were used for validation, and 435 (15%) images were used for testing. The model was executed for over 50 epochs. The graphs and outputs are presented in Section IV.

We used the following strategies to improve our drowsiness detection CNN model.

- (a) **Data Augmentation:** It transformation of an image to different angles. Flipping an image and rotating it at various angles are different data augmentation methods that ensure that transformations do not change the correct class. Data augmentation increases the number of images and the data size. They can be used for object recognition, speech recognition, etc. Data augmentation was used to train the model for 50 epochs, and the accuracy results were obtained. With data augmentation, we observed an improvement in the accuracy, as explained in Section IV.

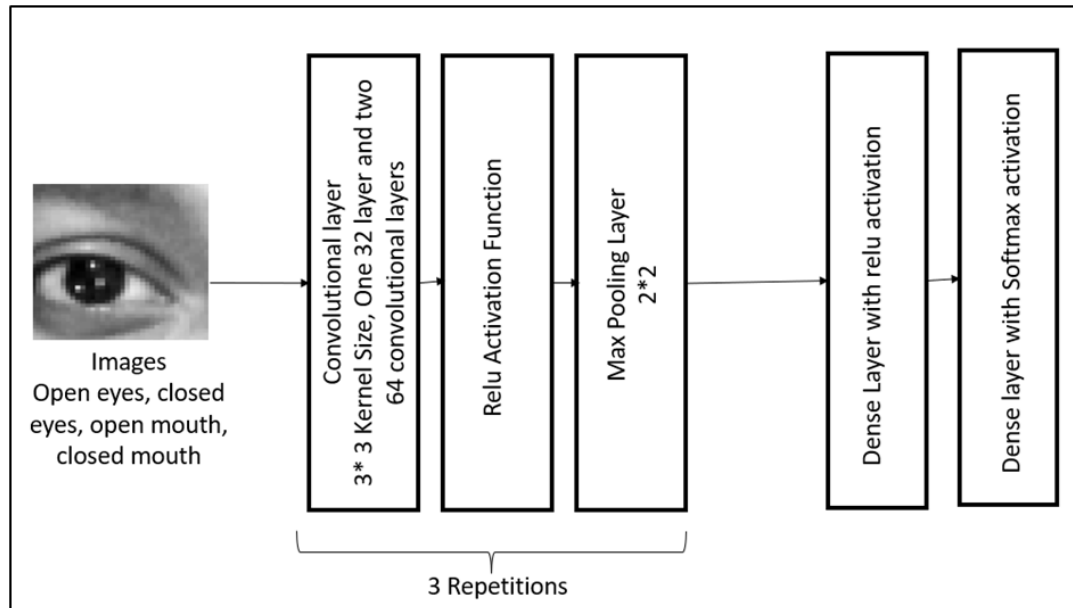


FIGURE 8. CNN architecture for drowsiness detection.

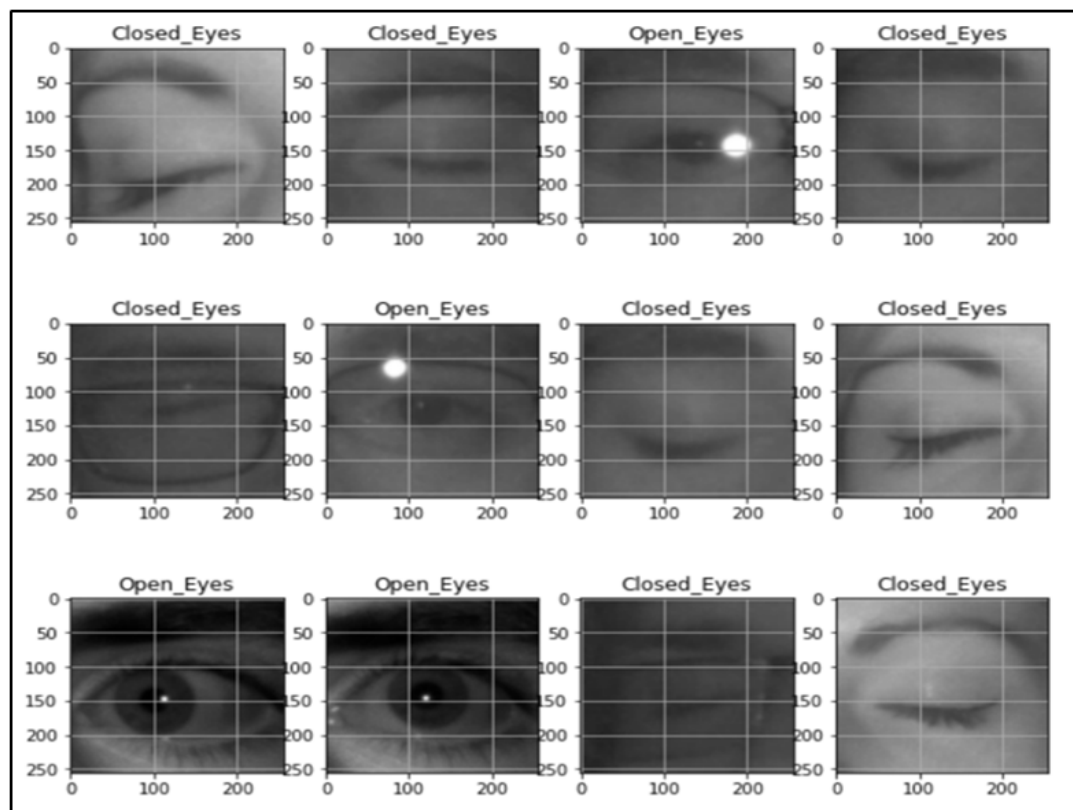


FIGURE 9. Images mapped on grid format.

(b) **L2 regularization:** We used L2 regularization to solve the optimization problem for our neural-network implementation. $[\min \tilde{J}(\vec{w}) \text{ subject to } \|\vec{w}\|^2 \leq s]$ approximately = $\min [\tilde{J}(\vec{w})] \dots \dots \dots (*)$, where

for every value of λ , an 's' exists such that the solution for (*) will give the same L2 regularization coefficients. The L2 regularization optimizes the weight of $\tilde{J}(\vec{w})$. The parameter 's' is inversely proportional to

λ . The coefficients are almost zero in L2 regularization. Three keyword arguments are used to implement the regularizer, which applies a penalty to the kernel of the layer. A bias regularizer applies a penalty to the layer bias and the activity regularizer. In our model, a value of 0.0001 is used as the regularizer. The model was trained, and it was observed that lower accuracy values were obtained compared with the basic model development. These graphs are explained in Section IV.

- (c) **Complex Neural Network:** While creating the sequential model, four convolutional layers consisting of 64 feature maps, 33 kernel size, activation function ReLU, max pooling with a kernel size of 22 and a dense layer with ReLU and SOFTMAX activation functions were defined for model development. A slightly more complex model was developed and trained for the open- and closed-eye datasets. The model was trained for 50 epochs with this change in the sequential network, and the results were obtained, as explained in Section IV.
- (d) **Dropout:** The dropout layer is another practical method for implementing a neural network in CNN architecture. Adding a dropout layer reduces the number of units and removes the non-output units. Dropout rates ranged from 0 to 1. This rate depends on the fraction of each link that is to be dropped. A rate of 0.4 was taken in the model. Three dropout layers are included after each convolutional and pooling layer. The model is implemented, and the results are presented in Section IV.
- (e) **Varying the split ratios of the training and validation datasets:** The model was initially trained for the four-label dataset. As mentioned in the first implementation, 80% of the dataset was used for training and the remaining 20% was used for validation. In this implementation, we used 90% of the dataset was used for training and 10% for validation. Thus, out of the total 2467 datasets, 2220 images were used for training and 247 were used for validation. The size of the dataset used for testing was 433. The model was trained using the same sequential model as done before, consisting of three convolutional layers of 32 and 64 feature maps, 3×3 kernel size, activation function ReLU, and three layers of Max Pooling with 2×2 kernel size followed by a dense layer with ReLU and Softmax activation functions.

2) SUPPORT VECTOR MACHINE (SVM)

The SVM is useful for solving both classification and regression problems. The support vectors are the nearest positive and negative points passing through negative and positive hyperplanes, respectively. SVM kernels convert the low dimension into a high dimension; therefore, splitting the positive and negative points is possible. The nonlinear decision boundary, a key feature of SVM, allows it to handle complex data distributions. SVM is expressed as $f(x) = \beta_0 + \sum_{i \in S} \alpha_i K(x, x_i)$, where S is the set of support vectors. An SVM kernel is a generalization of an inner

product. The kernel can be either a polynomial or radial kernel.

The SVM model was employed for both binary classification and multi-class classification tasks. It leverages kernel functions (such as polynomial or radial basis function kernels) to handle complex data distributions and nonlinear decision boundaries. We integrated the SVM model into the Convolutional Neural Network (CNN) architecture to significantly enhance classification performance. The last fully connected layer of the CNN was modified to include an SVM-based output layer. This adaptation allowed the model to use hinge loss (or squared hinge loss for multi-class problems) instead of traditional softmax-based loss functions, significantly improving its ability to distinguish between classes. The SVM model demonstrated its effectiveness as a lightweight classifier for drowsiness detection, especially in scenarios where computational resources are limited. It provided a baseline for comparing the performance of more advanced models like CNN and transfer learning, but its improved performance was awe-inspiring.

An SVM was used in the convolutional neural network to train the model. This is a perfect algorithm for object classification. Figure 10 shows the implementation of the SVM model. The MRL dataset chosen for its comprehensive collection of open- and closed-eye images was used to train the dataset. This dataset is particularly useful for training models for eye-state detection. The *Imagedatagenerator* was imported for data augmentation techniques to create more data images in memory. A horizontal flip was defined for augmentation, and the zoom range, shear range, and rescale values were mentioned. The training set defined the train dataset path, the target size of (64,64), the batch size of 32 and the class mode was defined as binary for two classes. If multiple classes exist, then the class mode is categorical. Before defining the test directory, the test image dataset is rescaled. Similar to the training dataset, the test dataset was defined with a target size of (64,64) and batch size of 32 and the class mode was defined as binary for classification and categorical for multi-class. It displays the total number of images in both the datasets. For the four-label dataset, 2467 train images and 433 test images belong to each of the four classes. For the two-label dataset, 61954 train images and 5028 test images belong to each of the four classes.

The convolutional and dense layers were imported from Keras. We implemented a CNN with 32 filters, a padding layer, a kernel size of three, a ReLU activation function, and two strides. A max-pooling layer is also defined. Another convolutional and max-pooling layer was added. A flattening layer was added, a fully connected layer of 128 units was added, and the activation function was defined as a ReLU. The crucial functionality was to add the last output layer as densely as possible with a defining kernel regularizer, where l2 norm with 0.01 value was defined inside the regularizer. The defined activation function was linear because a linear function was used for the binary SVM and a margin line was drawn between them to divide the two classes. For

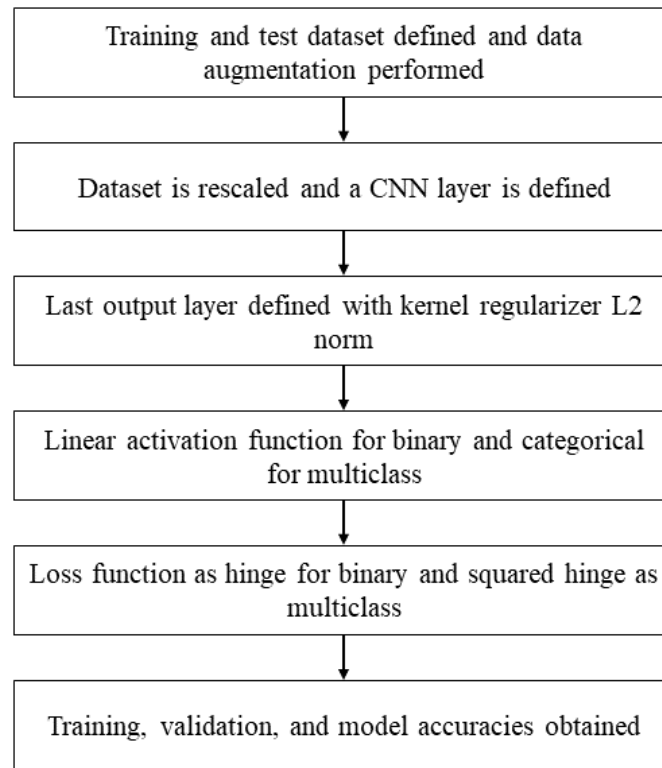


FIGURE 10. SVM model implementation.

multi-classification, a dense layer was defined, and because there were four classes, kernel regularizer l2 was defined as four and the activation function as softmax. A summary of the CNN is displayed and the loss function is compiled as a hinge. For multi-classification, the loss function is defined as a squared-hinge function. These two changes define the specific loss function and the regularizer. The final layer was automatically converted to the SVM layer if these two parameters were added, thereby demonstrating the adaptability of the proposed model to various tasks. The obtained accuracy and loss graphs are presented in Section IV.

C. TRANSFER LEARNING (TL) APPROACH FOR DROWSINESS DETECTION

In transfer learning, an existing model is used for training, instead of developing it from scratch. We usually start with 50% to 60% of the already trained models to train our model. The benefit of using the TL to train our model is that we do not need to train it from scratch. Simultaneously using TL to train the model, we must ensure that it is similar to the classification model being trained. For example, if TL is trained for a classification model with a similar task, such as classifying different types of animals, it can be reused for classifying open and closed eyes. This is because the features learned by the model for the animal classification task are likely to be helpful in our eye classification task. Transferring

learning is better than creating a customized model from scratch.

Different deep learning classifiers, such as MobileNet, GoogleNet, AlexNet, VGG19, and ResNet101 are popular for the ImageNet dataset. The ImageNet dataset contains thousands of images for approximately 1000 classes and 130 GB of data. These classifiers were trained using 1000 classes. Our study aimed to classify eyes into two groups: open and closed. It is a binary classifier. The TL uses the last fully connected layer, which is a key component in transfer learning. This layer contained approximately 1000 nodes, each representing a class in the original dataset; however, only two nodes were required for our binary classification task. All weights were used using a deep learning architecture. MobileNet was used to implement this model.

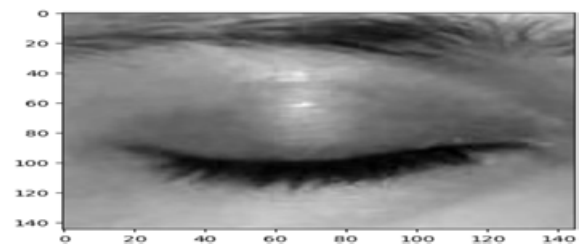


FIGURE 11. Closed eye image.

The images were read and plotted as shown in Figure 11. The size of each image was 145×145 . The training dataset

and list of classes were defined. The first image was displayed and its size was modified to 224. Figure 12 presents the pseudo-algorithm of the TL implementation. All the images were read and converted into an array of data and labels. The complete image set is transformed into an array. A class number was assigned, which signified the labels for the closed and open eyes. The training length was checked for 2467 images. Random shuffling is performed to avoid over-fitting. X and Y are created to append the features and labels.

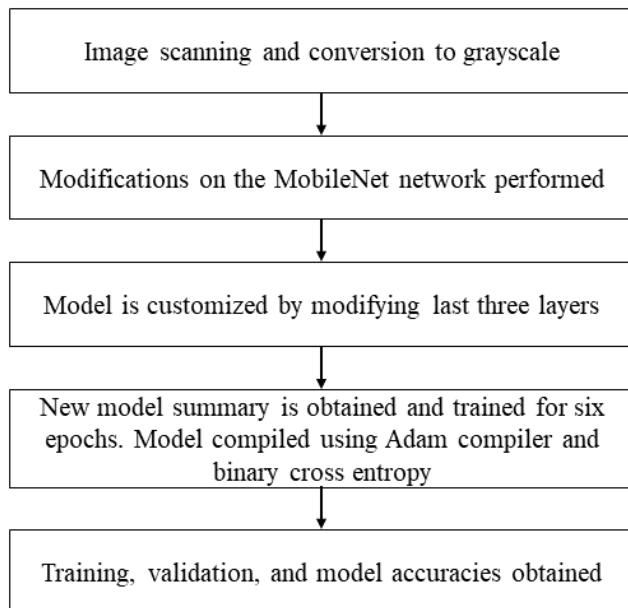


FIGURE 12. Transfer Learning implementation.

The training data were stored as features and labels. The features are stored in X and the labels in Y. The shape of X is displayed (2467, 224, 224, 3). It contains 2467 images of size 224×224 and 3 indicates RGB. All images were converted to gray into RGB images because the Imagenet contains RGB images. The Imagenet works only with (224, 224, 3). Hence, it was converted, and the data were normalized. The data and labels were transformed into a Numpy array. The MobileNet model was selected from Keras because of its light weight.

The architecture of the MobileNet network and the network modifications are shown in Figure 13. A summary is displayed, which shows all layers present in the MobileNet model. The base output layer was considered, and by dropout of the last three layers, the final output of the dropout layer was considered the output. We added a dense layer for one node, either 0/1 for the open and closed eyes. For the final output, a sigmoid layer is added to the classification model. After loading the dropout layer and new model, three layers were added.

In the new model, three additional layers are added and concatenated. The total number of parameters was reduced because the last convolutional layer that was removed contained more parameters. The total number of trainable and non-trainable parameters was also reduced. The activation

function was set as a sigmoid function, which is suitable for binary classification. The new model was compiled using the Adam optimizer and binary cross-entropy. The model was trained for six epochs. The accuracy starts at 70%, saving time, which is an advantage of transfer learning. This model was saved. The TL method was implemented on both datasets and the results were obtained. Two other changes in the model were made: one was by changing the training and validation split ratio and the other was by changing the activation function. The obtained graphs and accuracies are presented in Section IV.

D. CHECKING THE NETWORK FOR PREDICTIONS

The prediction is verified when a new image is loaded. Faces were captured using a webcam, and facial landmarks were detected using OpenCV. These values were calculated using the EAR and MAR algorithms. Based on the prediction of each live image, the driver was alerted when the EAR or MAR value exceeded a certain limit. A test image was loaded, reshaped, and converted into RGB. The prediction was verified using a model developed using this input. If the predicted value was approximately 16, it was considered positive. The loaded image was an open-eye image and a positive prediction value was obtained. A negative predictive value was obtained when a closed-eye image was loaded.

Figure 14 shows the test image used to verify the unknown images. It is a colored image in which the face and eyes are detected. The image was converted into gray as the model was trained using gray images. The eyes were detected and a rectangle was drawn around the eye region, as shown in Figure 15. The image was converted into RBG and resized. The prediction results are shown in Figure 16. The eye area was cropped and the predictions were checked.

A real-time video demonstration was performed using live feed from a camera. The face was detected using OpenCV, and eye coordinates were obtained from the images. The predictions were verified by using a live video of the trained model. If the prediction was greater than 0, the eyes were open; if the prediction was less than 0, the eyes were closed. A counter was used to check the frequency with which the eyes were closed. If the counter value was exceeded, a sleep alert message was displayed and a buzzer sounded.

IV. EXPERIMENTAL RESULTS & ANALYSIS

We used performance metrics such as the confusion matrix, precision, recall, and F1-score, to evaluate the performance, accuracy, and loss of our model. The training accuracy was defined as the accuracy of the model obtained using the same images used for training. Test accuracy is the accuracy with which the trained model can identify new or independent images that are not used for training. The validation accuracy is the accuracy obtained from the training of the model on the validation dataset. A confusion matrix is a table that shows the model's actual performance, that is, how well it can classify images. The number of actual and incorrect predictions was analyzed using a confusion

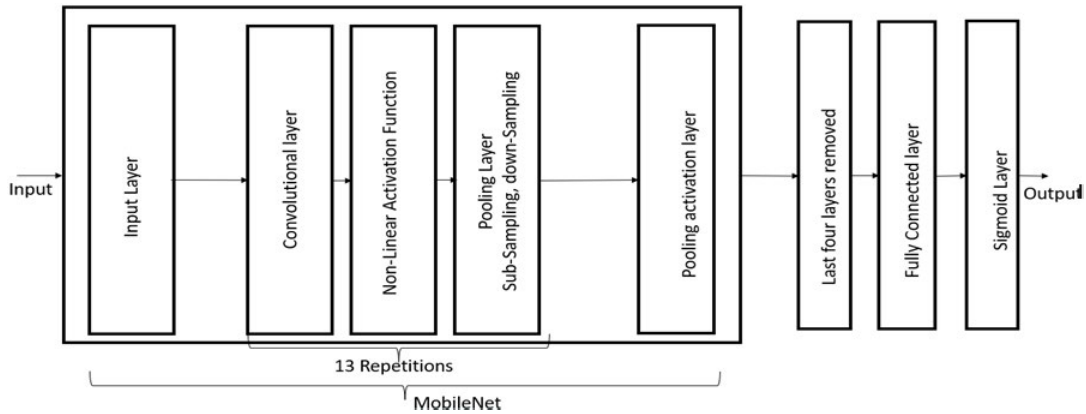


FIGURE 13. Deep Learning model development using transfer learning.

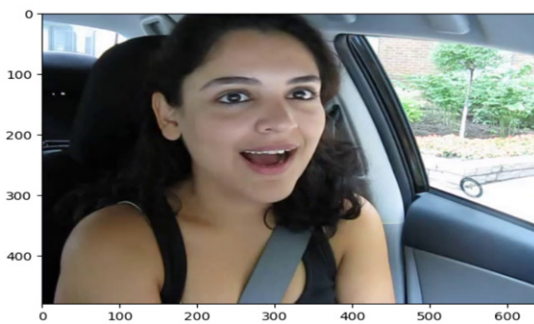


FIGURE 14. Test image.

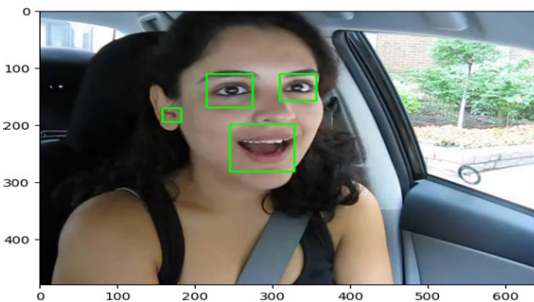


FIGURE 15. Boxed image.

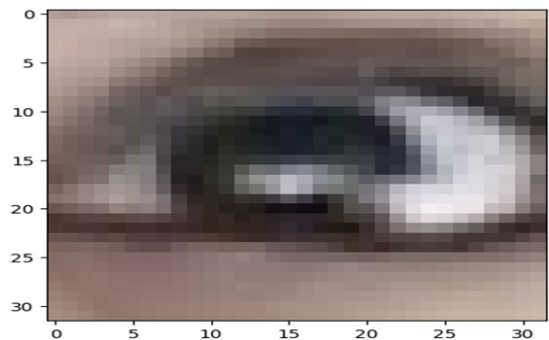


FIGURE 16. Eye area.

matrix. Precision represents the percentage of correctness of positively classified classes. Recall is another performance matrix that indicates the number of times that a specific class occurs. F1-score was the harmonic mean of the precision and recall values.

We used Python 3.8 version to conduct the experiments and test the performance of our model. We used libraries such as Dlib to identify frontal facial landmarks. Dlib also contains built-in functions for machine learning that can be used for image processing. We also used the Numpy library, which is a prerequisite for the Dlib library, and Scipy to calculate the distance. We included the OpenCV library in our study for video streaming and frame processing, and the Imutils

library was required for the OpenCV function. Additionally, we conducted experiments using Google Collab and built-in GPUs using the Jupyter Notebook.

The experimental results are presented in four subsections. Section IV-A explains the results of EAR, MAR, and the combined implementation. The results of the machine learning models developed using CNN and SVM are presented in Section IV-B. Section IV-C presents the TL methodology results. Finally, we provide detailed model comparisons in Section IV-D.

A. PERFORMANCE OF REAL-TIME DROWSINESS DETECTION INDICATORS

In this experiment, we detected the complete face in real-time by dividing it into right, left, top, and bottom. Once the face was detected, a rectangular box was drawn for the face detection, as shown in Figure 17.

We used this predictor to identify landmarks. The complete face has numbering similar to that of an array. The eye region also contains specific landmarks for detection. Separate numbering was performed in the left and right eyes. When the left and right eyes were closed for more than a specific time, the EAR ratio was less than 0.21. Figure 18 shows that the sleep state was detected as the output.

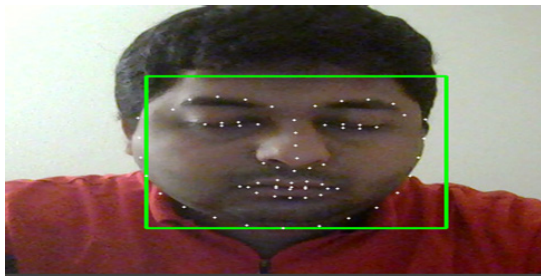


FIGURE 17. Complete face detection.



FIGURE 18. Sleeping face image.

Based on the eye position, a drowsy state was detected as an output when the EAR ratio is between 0.21 and 0.25, as shown in Figure 19.



FIGURE 19. Drowsy face image.

The active face was detected based on the open-eye state, where the EAR ratio exceeded 0.25, as shown in Figure 20.

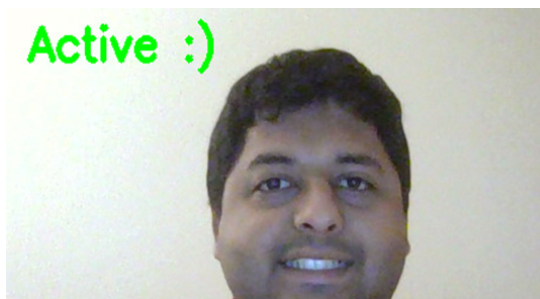


FIGURE 20. Active face image.

We used the MAR ratio for yawn detection in real-time. Facial landmarks were detected as shown in Figure 21.

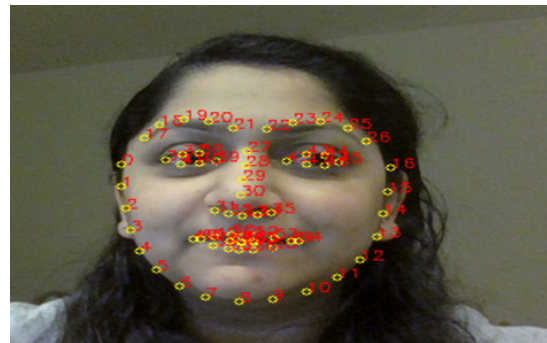


FIGURE 21. Facial landmarks.

Yawning was displayed when the distance between the lips was greater than 15, and the driver was alerted. The yawn count was recorded as shown in Figure 22.



FIGURE 22. Yawning detection.

The hybrid EAR and MAR algorithms were used for facial landmark detection, as shown in Figure 21. The hybrid ratio was computed to monitor the mouth and eye positions in real-time. Based on mouth and eye positions, the EAR ratio, yawn count, and drowsiness alert are displayed as shown in Figures 23.

B. CUSTOM CNN AND SVM-BASED PERFORMANCE RESULTS

1) CNN RESULTS

The model is trained using several methods. It starts with essential model development, followed by multiple improvement strategies such as data augmentation, L2 regularization, dropout, varying the training and validation set ratios, and a slightly complex model.

- 1) **Basic CNN Model:** This model was prepared for a four-label dataset. We divided the 2467 image dataset into 80%, that is, 1973 images for model training, and the 20%, that is, 494 images, for validation. The test dataset contained 433 images. The model was then trained for 50 epochs. We obtained 100% training accuracy and 94.91% test accuracy, as shown in Figure 24. The graph shows the accuracy and loss of training and validation. Accuracy was plotted along the X-axis and epochs on the Y-axis. A considerable training accuracy was obtained.



FIGURE 23. EAR and MAR hybrid output.

The training and validation loss graphs show that the training loss decreased with increasing time, and the validation loss first decreased and then increased. Thus, this model may be under-fitted. The distance between the training and validation losses was also high.

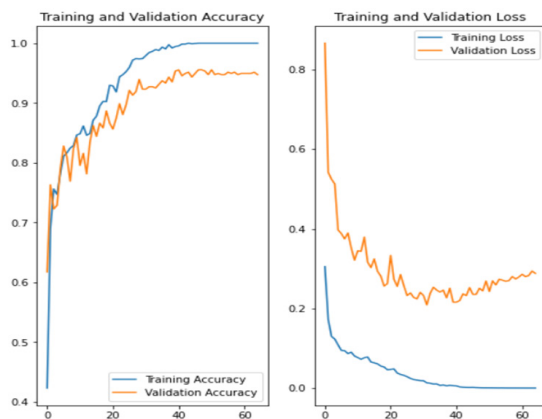


FIGURE 24. Accuracy and loss curves for basic CNN model.

Figure 25 shows the confusion matrix with eyes closed, eyes open, yawns, and no-yawns for the CNN model. As shown in the confusion matrix, most images were correctly predicted. For instance, 13 images were predicted to have no yawns but were actually yawns. Four images were predicted to be closed; however, they were open. One image was predicted to be open, but it was closed. Similarly, three images were predicted to be yawns, but no-yawns were predicted. Table 2 shows the CNN classification results. This indicates that the precision, recall, and f1-score values were close to one. In the case of a recall, it shows that overall, 95% of the truly positive images were classified as positive correctly. The predictions for 95% of the images were correctly classified as positive. The overall avg accuracy is approximately 0.95.

2) **Data augmentation results:** The model was trained for 50 epochs, and the accuracy results were obtained,

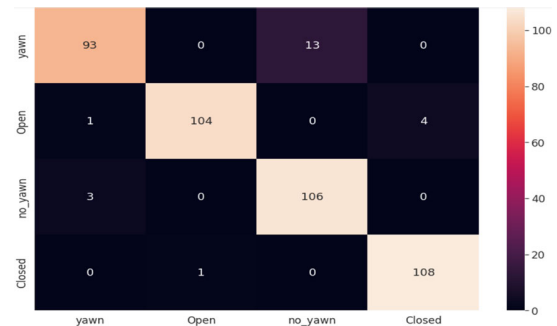


FIGURE 25. Confusion matrix.

TABLE 2. Classification report.

	precision	recall	f1-score	support
yawn	0.96	0.88	0.92	106
open	0.99	0.95	0.97	109
no_yawn	0.89	0.97	0.93	109
closed	0.96	0.99	0.98	109
accuracy			0.95	433
macro avg	0.95	0.95	0.95	433
weighted avg	0.95	0.95	0.95	433

as shown in Figure 26. The X-axis indicates the number of epochs, and the Y-axis shows the accuracy. A training accuracy of 100% and a validation accuracy of 96.35% were obtained. There was an improvement in the accuracy of data augmentation. Validation loss decreased and then slightly increased. Training loss decreased over time. The results were slightly better compared to the basic model.

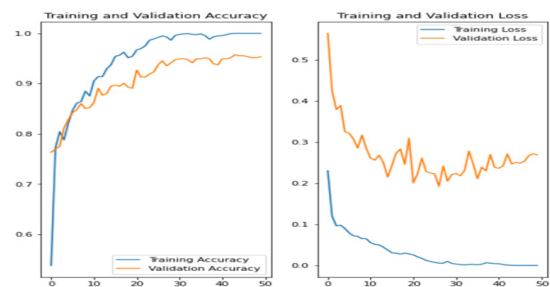


FIGURE 26. Data augmentation accuracy and loss graph.

- 3) **L2 regularization results:** The model was trained, and it was observed that lesser accuracy values were obtained compared to the essential model development. Overall training accuracy of 98.12% and test accuracy of 91.68% was obtained. Figure 27 presents the training and validation accuracy and loss. It was seen that the gap between the train and the validation loss was comparatively greater. The validation loss values were higher than the training loss values. L2 regularization has lower results than data augmentation and the basic model results.
- 4) **Complex neural network result:** Figure 28 shows that high training and validation accuracy was obtained.

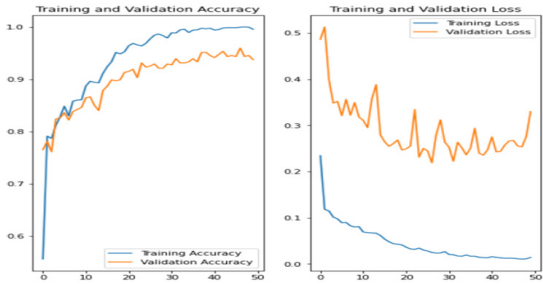


FIGURE 27. L2 regularization accuracy and loss graph.

Training and validation loss graphs where the X-axis is the number of epochs and the Y-axis is the accuracy. The gap between the training and the validation loss was significantly less. There are some improvements in the model compared to the basic model development. Better results were obtained compared to the basic model, L2 regularization, and data augmentation model results.

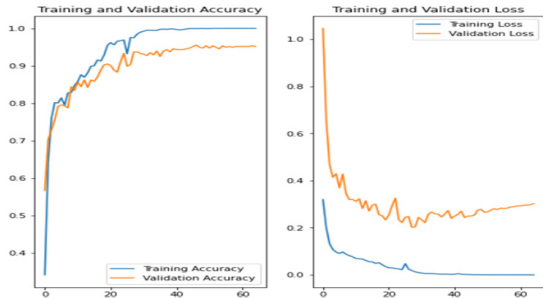


FIGURE 28. Complex neural network accuracy and loss graph.

From the confusion matrix in Figure 29, it can be observed that eight images were incorrectly predicted as no yawn instead of being detected as a yawn. This is an improvement over that of the base model. The other parts and number of mispredicted images were reduced and performed better overall.

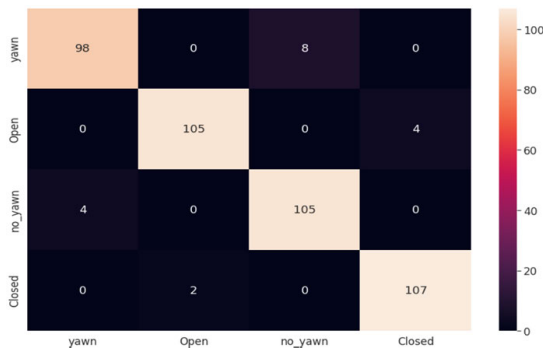


FIGURE 29. Complex neural network confusion matrix.

Table 3 shows the results of the classification report. Precision, recall, and F1-score values were $\geq 92\%$. In terms of recall, 96% of truly positive images were

correctly classified as positive. The prediction of 96% of the images were correctly classified as positive. An overall accuracy of 96% is achieved.

TABLE 3. Complex neural network classification report.

	precision	recall	f1-score	support
yawn	0.96	0.92	0.94	106
open	0.98	0.96	0.97	109
no_yawn	0.93	0.96	0.95	109
closed	0.96	0.98	0.97	109
accuracy			0.96	433
macro avg	0.96	0.96	0.96	433
weighted avg	0.96	0.96	0.96	433

5) **Dropout results:** For the dropout model, we achieved a training accuracy of 99.54% and a test accuracy of 96.35%. The validation accuracy also performed better, as shown in Figure 30. The X-axis indicates the number of epochs, and the Y-axis shows the accuracy. The figure shows that the training and validation loss have a minimal gap between them and that a better model was obtained.

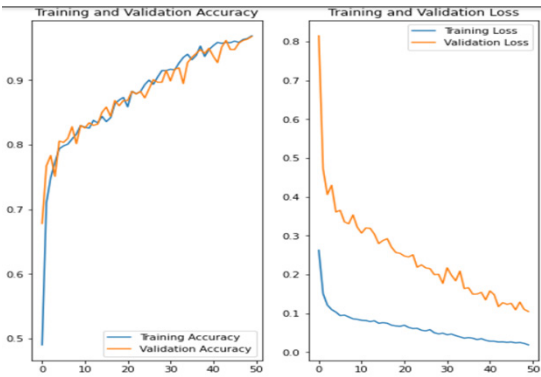


FIGURE 30. Dropout accuracy and loss graph.

A confusion matrix is shown in Figure 31. It was seen that seven images were mispredicted, such as no yawn instead of a yawn. There is comparatively an improvement over the basic CNN model, with 13 incorrect predictions reduced to seven.

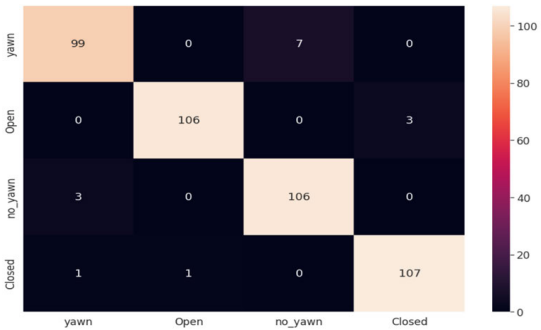


FIGURE 31. Dropout confusion matrix.

An overall accuracy of 0.96 was obtained, as shown in Table 4. It was seen that the precision, F1 score, and recall obtained all were 0.94 or above. In the case of a recall, it shows that overall, 96% of the true positive images were classified as positive correctly. The prediction of 96% images was correctly classified, which were classified as positive.

TABLE 4. Dropout classification report.

	precision	recall	f1-score	support
yawn	0.94	0.95	0.94	106
open	0.97	0.96	0.97	109
no_yawn	0.95	0.94	0.95	109
closed	0.96	0.96	0.96	109
accuracy			0.96	433
macro avg	0.96	0.96	0.96	433
weighted avg	0.96	0.96	0.96	433

6) **Varying the training and validation dataset sizes:** We trained the model for 50 epochs, and a training accuracy of 100% and a test accuracy of 96.7% were obtained, as shown in Figure 32. The X-axis indicates the number of epochs, and the Y-axis shows the accuracy. It was the best model compared to the other developed models due to the highest training accuracy. The gap between the training and the validation loss was also less.

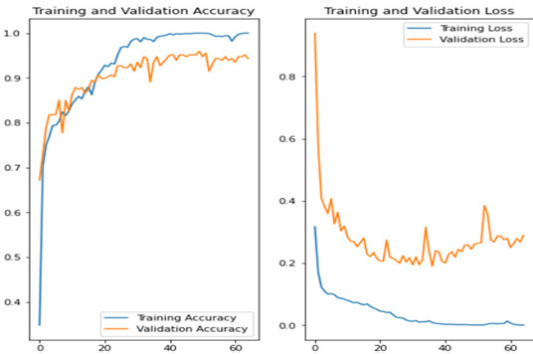


FIGURE 32. Accuracy and loss graphs for varying dataset size.

As shown in Figure 33, the confusion matrix shows an overall improvement compared to the confusion matrix obtained from the basic, dropout, and complex models. The basic model had 13 images detected incorrectly, such as no yawn instead of a yawn. Here, the value was reduced from 13 to 6 images, detected incorrectly as no yawn instead of a yawn. 100 images out of 106 were correctly detected for the overall yawn images. Table 5 shows the values of the precision, recall, F1 score, and support. It can be seen that all values were approximately 0.95 or above. In the case of recall, it shows that 97% of truly positive images were classified as positive. The predictions for 97% of the images were correctly classified as positive. The overall accuracy improved to 0.97.

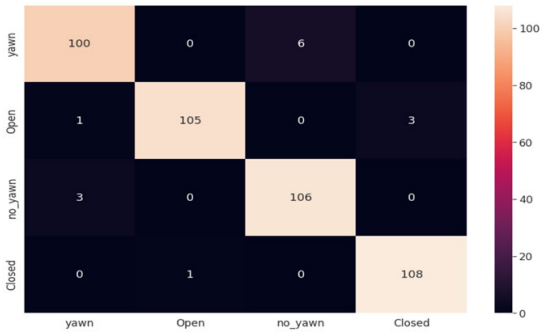


FIGURE 33. Confusion matrix for varying dataset size.

TABLE 5. Classification report for varying dataset size.

	precision	recall	f1-score	support
yawn	0.96	0.94	0.95	106
open	0.99	0.96	0.98	109
no_yawn	0.95	0.97	0.96	109
closed	0.97	0.99	0.98	109
accuracy			0.97	433
macro avg	0.97	0.97	0.97	433
weighted avg	0.97	0.97	0.97	433

Figure 34 summarizes the training and testing accuracies for all the deep learning models developed. It can be seen that the best model was obtained by varying the training and validation ratios. 100% training accuracy was obtained for the basic model, the model with data augmentation, slightly complex model, and varying training and validation ratios. The models with varying training and validation ratios yielded the highest test accuracy. However, for all models, an accuracy $\geq 90\%$ was obtained for both test and training.

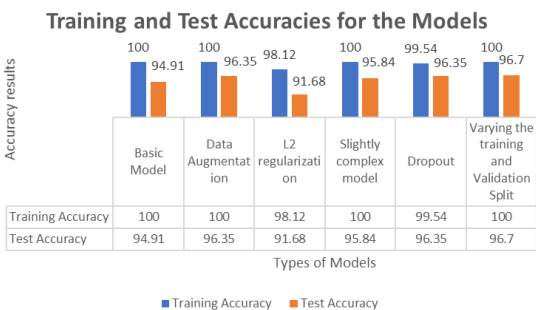


FIGURE 34. Test and training accuracies for the models.

Figure 35 compares the percentage of correct predictions for the yawn images only. 88% were correctly predicted for the basic model, 92% for the slightly complex model, 93% for the dropout model, and 94% for the model developed by varying the training and validation split ratios. Comparing the correctly predicted confusion matrix, the model developed by varying the training and validation split ratios performed better than the other models.

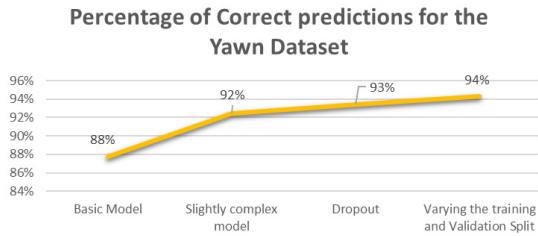


FIGURE 35. Percentage of correct prediction for yawn dataset.

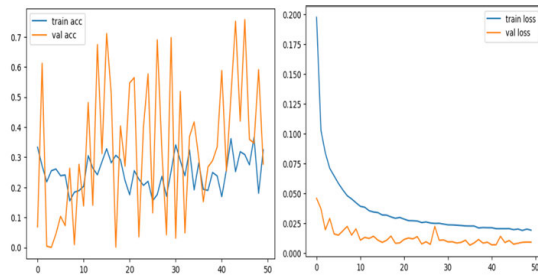


FIGURE 36. Accuracy and loss SVM result for two label MRL dataset.

2) SVM RESULTS

We trained the model for 50 epochs for the MRL dataset and for two and four labels. Figure 36 and 37 show the accuracy and loss results for two- and four-label MRL datasets, respectively. The x-axis indicates the number of epochs, and the y-axis shows the obtained accuracy. The training and validation accuracies are extremely low for the MRL dataset. According to the training and validation loss graphs, both the values decreased with time. The training and validation accuracies were significantly better, with approximately 92.26% and 91.92% training and validation accuracies, respectively. The training and validation loss curves also decreased. Higher accuracy could be obtained if the model was trained for more epochs.

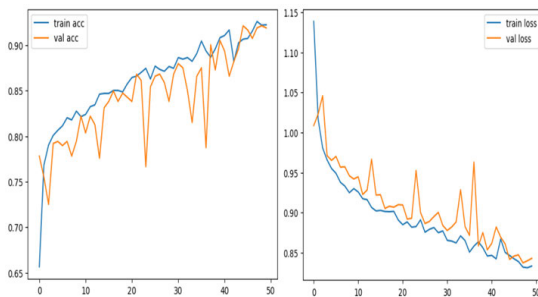


FIGURE 37. Accuracy and loss SVM result for four label MRL dataset.

C. TRANSFER-LEARNING RESULTS

- 1) **Basic Model:** We trained the 4-label dataset for 50 epochs and obtained a training accuracy of 46% and validation accuracy of 34.41% was obtained. The same model with two labels was trained using the MRL dataset. The results are shown in Figure 38, and the same

test was performed using several images from the MRL dataset. Approximately 61,954 images were obtained. The model was trained for six epochs, and a training accuracy of 99.44% and a validation accuracy of 99.10% were obtained.

A graph of the training and validation graphs is shown in Figure 38. The y-axis shows the accuracy and the x-axis indicates an increase in the number of epochs. The training accuracy was high. When the validation loss was observed, the changes were random. The training loss decreased continuously. It was observed that transfer learning was more effective when the dataset size was large. In this case, the number of images in the first dataset was low; therefore, transfer learning was ineffective. With an increase in dataset size, transfer learning is more effective. Transfer learning was more effective for multiple classes with more data. In this case, as it was a binary classification, the transfer learning MobileNet model performed well, but it would be beneficial for a larger dataset with many classes, as the network is much more complex and consists of many convolutional neural networks.

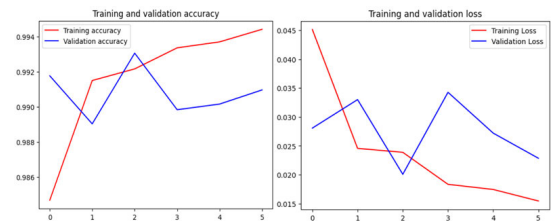


FIGURE 38. Accuracy and loss for transfer learning basic model.

- 2) **Varying the training and validation split ratio model results:** The training and validation splits were varied and modeled to improve the performance of the model. 30% of the dataset was used for validation, and the remaining 70% was used for training. The model was trained for six epochs. A training accuracy of 99.47% and a validation accuracy of 98.89% were obtained. The graphs of the training and validation loss and accuracy are shown in Figure 39. The Y-axis shows the accuracy, and the X-axis indicates the number of epoch increases. It was observed that with the change in the splitting of the training and the validation, the training accuracy is improved compared to the basic model results, but the validation accuracy is decreased.

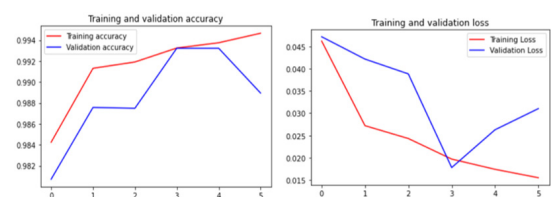


FIGURE 39. Accuracy and loss for transfer learning varying ratio model.

3) **Using ReLU activation function model results:** We developed a model similar to the basic one described earlier but with a ReLU activation function that was used instead of the sigmoid in the final output layer. The model was trained for six epochs, and the accuracy results were obtained. It is observed that a training accuracy of 92% and validation accuracy of 96.63% was obtained. The graphs in Figure 40 were plotted for the training and test accuracies, validation, and loss. The Y-axis shows the accuracy, and the X-axis indicates the number of epoch increases. It was observed that with the change in the splitting of the training and the validation, training accuracy improved, but the validation accuracy decreased. It was also observed that there was a massive dip in the validation accuracy. The training accuracy values vary continuously. The training and the validation loss graph changes were also random. This model development was not a good approach. Using the sigmoid activation function performs better for the binary image classification model. As it's a simple problem containing open and closed eyes images, having a sigmoid activation function is a better option.

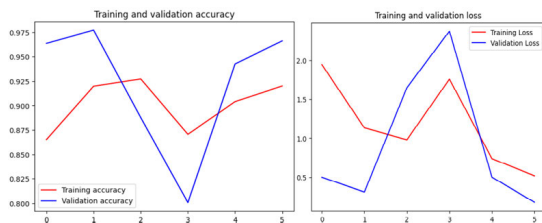


FIGURE 40. Accuracy and loss for transfer learning activation function model.

Figure 41 shows a comparison bar chart of the consolidated results of the various models developed using transfer learning. The y-axis shows the accuracy obtained, and the x-axis shows the different methods developed using transfer learning. The ReLU model did not perform as expected. The initial transfer learning model developed performed well. By varying the training and validation split ratios, good performance was obtained, but slightly less than the initial model development. The initial transfer learning model performed better than the other two models, with better accuracy.

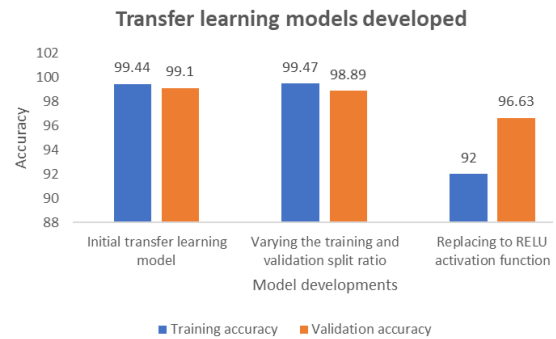


FIGURE 41. Transfer learning model development comparison.

varying the ratios of the training and validation dataset sizes. We achieved better training and validation accuracy with the CNN and SVM. The training and validation accuracies for CNN were 100% and 94%, whereas for SVM, they were 92.26% and 91.92%, respectively. Potential changes in the network, a more complex model, or running it for more epochs could have resulted in better accuracy. Compared to CNN, SVM took almost one hour less to execute 50 epochs. Additionally, the table presents the results obtained for the 2-label MRL dataset. We achieved higher training and validation accuracies using the CNN and transfer learning models. The CNN was trained for 50 epochs, and the transfer learning model was trained for six epochs. We achieved training and validation accuracies of approximately 100% for the CNN and transfer learning models. The transfer learning model took two hours less to train than the CNN model. Therefore, transfer learning is a better approach, as the accuracy increases from 60% from the first epoch.

Table 7 presents a formal comparison between individual EAR and MAR algorithms and the implementation of the hybrid EAR/MAR algorithm. Based on the obtained results, it was observed that the hybrid algorithm made the model more robust and accurate than the individual algorithm. The hybrid algorithm includes more drowsy detection scenarios. However, a hybrid algorithm requires extra time for prediction and its implementation is more complex than that of an individual algorithm.

Table 8 shows the results of the CNN/SVM and transfer learning models. Based on these results, it can be inferred that more time was spent training the CNN/SVM model developed from scratch than on transfer learning. Transfer learning yielded better accuracy with a lower number of epochs. The CNN/SVM model provides better accuracy but requires more epochs. Several trials and errors must be performed when using transfer learning. With transfer learning, we need to check whether the model we are reusing fits our dataset, and how flexible it is to be improvised. However, a few criteria need to be examined. In our study, we modified the last four layers of the pre-trained model to suit our specific dataset and application. This process involves adjusting the weights and biases of these layers to

D. COMPARATIVE ANALYSIS AND DISCUSSION

Table 6 lists the various CNN models developed based on their training and test accuracies. The model developed by varying the training and validation split ratios yielded the highest training and testing accuracy. The basic data augmentation and complex models achieved 100% training accuracy and approximately 95% testing accuracy, respectively. The table also shows the result obtained for the 4-label dataset trained for 50 epochs. It can be seen that the CNN's maximum accuracy was improved; that is, the model was trained by

TABLE 6. Accuracy obtained with the various CNN methods.

Types of models developed	Training Accuracy (%)	Test/Validation Accuracy (%)
Basic Model	100	94.91
Data Augmentation	100	96.35
L2 regularization	98.12	91.68
Complex model	100	95.84
Dropout	99.54	96.35
Varying training and validation split	100	96.7
CNN with improvement (50 epochs, 4-label dataset)	100	94
SVM (50 epochs, 4-label dataset)	92.26	91.92
CNN with improvement (50 epochs, 2-label MRL dataset)	100	99.7
Transfer Learning (6 epochs, 2-label MRL dataset)	99.44	99.1

TABLE 7. Individual versus hybrid EAR/MAR algorithm comparison.

Individual EAR and MAR algorithm	Hybrid EAR-MAR algorithm
Less time for the predictions	More time for the predictions
Less robust and has lower accuracy compared to a hybrid algorithm	More robust and accurate
Does not cover all scenarios for drowsiness detection	Covers more scenarios for drowsiness detection
Individual EAR and MAR algorithms are simple to implement	EAR-MAR hybrid algorithm is complex to implement

better fit the data. The model must be customized according to the size and application of the dataset. However, its flexibility is comparatively lower than that of a model developed from scratch because it can be changed as needed.

1) LIVE PREDICTION RESULTS AND COMPARISON BETWEEN EXISTING MODELS AND THE PROPOSED MODEL

We created a live fake drowsy environment using a webcam, as shown in Figure 42. The figure shows that our model successfully detected drowsy conditions from facial landmarks identified through the live webcam feed. When the eyes were closed, model predictions were negative. The eyes were opened when the prediction was positive. When the prediction was negative, a drowsiness alert was displayed along with an alarm. The message is displayed when the person yawns a lot, and the driver is alerted. The EAR values and yawn counts are presented in the figure.

**FIGURE 42.** Live webcam face drowsiness prediction result.

Table 9 presents a performance comparison between the proposed models based on the results and the existing models discussed in the literature survey section. As shown in the table, previous studies have used CNN, multi-scale CNN, 4D CNN, IoT-based systems, graph NN, SVM, transfer learning,

and LSTM models. These studies achieved an accuracy of $\geq 90\%$. For our proposed models 1 and 2, we achieved 100% and 99.4% training accuracies, respectively. We achieved approximately 99% validation accuracy for models 1 and 2. With proposed model 3, we achieved 92.26% training accuracy and 91.92% validation accuracy.

V. LIMITATIONS & FUTURE RESEARCH DIRECTION

Despite these advancements, the system is not without limitations. One significant limitation is its reliance on facial landmarks, which assumes ideal conditions, such as proper lighting and unobstructed views of the driver's face. Poor lighting, occlusions (e.g., sunglasses or hands covering the face), or extreme head poses could hinder accurate detection. Additionally, the system's performance might degrade in cases where drivers have unique facial features, such as heavy eyelids or beards, which could interfere with calculating EAR and MAR values. Another challenge lies in the computational requirements of the hybrid EAR-MAR algorithm, which may pose difficulties for deployment in resource-constrained environments, such as older vehicles or embedded systems with limited hardware capabilities. Furthermore, while effective, the datasets used in this study may not fully represent the diversity of real-world driving conditions. Expanding the dataset to include more varied scenarios and demographic factors will improve the system's generalizability.

This research has broader applications beyond road safety. The methodology can be adapted for monitoring heavy machinery operators, workers in hazardous environments, and professionals handling sensitive tasks in industries such as chemical laboratories and manufacturing plants. By laying the groundwork for future innovations, this study highlights the importance of developing smarter, more effective solutions to combat driver fatigue.

TABLE 8. CNN/SVM versus transfer learning comparison.

CNN/SVM	Transfer Learning
More time for training the model as it is developed from scratch	Less time for training as we are reusing an already existing model
Better accuracy obtained after training for a greater number of epochs	Better accuracy was obtained with training for a lower number of epochs
Performance improvement takes more time	Performance improvement takes less time
Model is flexible and can be customized as per our need	Model is not flexible. It depends on which model is reused and whether it fits our dataset

TABLE 9. Proposed vs. previous work.

Reference Number	Topic description	Accuracy obtained (%)
[22]	Leveraging transfer learning for vehicle safety using transfer learning (MRL dataset)	97
[27]	Eye aspect ratio using Raspberry Pi	90
[32]	C-LSTM for detecting drowsiness	95 - 97
[33]	ECG classification using LSTM	99.26
[34]	Transfer learning, pre-trained CNN AlexNet model	97.89
[35]	SVM for discrete wavelet transform	98.8
[36]	CNN for cardiac detection	97.42
[37]	CNN for myocardial function	93.53
[40]	Multi-scale convolutional layer	98.18
[41]	Heart rate, fibrillation detection	97.83
[44]	4D CNN model for eye state categorization	97.53
[45]	CBAM VGG16 deep learning architecture to improve the classification performance of driver distractions	98.65 classification accuracy for camera 1 & 97.85 for camera 2 dataset
[46]	IoT-based embedded system using a Raspberry Pi	96 - 99
[47]	Improved three sub-networks of the multitask CNN: P-, R-, & O-Net	93.65
[48]	A prior knowledge-guided multi-information graph convolutional network	94.78 & 97.52 for two different dataset
[49]	Leveraged CNN and LSTM networks using a WACHSens dataset collected from both manual and automated driving modes, encompassing rested and fatigued states	96
Proposed Model 1	CNN model with improvement	99.7 for 2-label dataset & 94 for 4-label dataset
Proposed Model 2	Transfer learning model	99.1 for 2-label dataset
Proposed Model 3	SVM	91.92 for 4-label dataset

This research represents a meaningful step toward creating safer roads through technology. By advancing state-of-the-art drowsiness detection systems, we aim to significantly reduce accidents caused by driver fatigue and contribute to a safer, more alert driving experience. This potential for positive change should inspire the audience and instill a sense of hope for the future. Future work will focus on refining the system, addressing current limitations, and exploring additional features to enhance its reliability and effectiveness.

VI. CONCLUSION

Driver's drowsiness, a significant contributor to road accidents, has been effectively addressed with our practical and user-friendly Driver Drowsiness Detection System. This system, leveraging advanced algorithms and machine learning techniques, can detect real-time fatigue. It integrates hybrid EAR and MAR algorithms with state-of-the-art models like CNN, SVM, and Transfer Learning. Using non-intrusive facial landmark detection via a webcam eliminates the need for wearable sensors or additional hardware, making it a convenient and accessible solution.

This study makes a valuable contribution to the field by introducing a robust framework that combines traditional computer vision techniques with modern deep learning approaches. Transfer learning is particularly noteworthy, as it reduces computational demands while maintaining high performance, making the system scalable for a wide range of

real-world applications. Additionally, the hybrid EAR-MAR algorithm offers a more comprehensive solution for detecting drowsiness, addressing a wider range of scenarios than systems that rely solely on individual features like eye closure or yawning. The novelty of this research is evident in its holistic approach to driver drowsiness detection. By combining hybrid EAR-MAR algorithms with state-of-the-art machine learning models, we have developed a system that outperforms traditional methods in accuracy and reliability, instilling optimism and a forward-thinking mindset in the audience.

The experimental results demonstrate the effectiveness of our approach and instill confidence in the system's ability to detect drowsiness reliably. Furthermore, the hybrid EAR-MAR algorithm proved to be more robust and accurate than individual EAR or MAR implementations, covering more scenarios of drowsiness detection.

ACKNOWLEDGMENT

The authors express their gratitude to San José State University for their support and provision of resources that facilitated this study.

REFERENCES

- [1] (2017). *Drowsy Driving in Fatal Crashes, United States*. Accessed: Feb. 3, 2025. [Online]. Available: <https://aaaafoundation.org/wp-content/uploads/2024/03/202304-AAAFTS-Drowsy-Driving-Countermeasures.pdf>

- [2] G. Li, B.-L. Lee, and W.-Y. Chung, "Smartwatch-based wearable EEG system for driver drowsiness detection," *IEEE Sensors J.*, vol. 15, no. 12, pp. 7169–7180, Dec. 2015, doi: [10.1109/JSEN.2015.2473679](https://doi.org/10.1109/JSEN.2015.2473679).
- [3] L. Pauly and D. Sankar, "Detection of drowsiness based on HOG features and SVM classifiers," in *Proc. IEEE Int. Conf. Res. Comput. Intell. Commun. Netw. (ICRCICN)*, Kolkata, India, Nov. 2015, pp. 181–186, doi: [10.1109/ICRCICN.2015.7434232](https://doi.org/10.1109/ICRCICN.2015.7434232).
- [4] I. A. Fouad, "A robust and efficient EEG-based drowsiness detection system using different machine learning algorithms," *Ain Shams Eng. J.*, vol. 14, no. 3, Apr. 2023, Art. no. 101895, doi: [10.1016/j.asej.2022.101895](https://doi.org/10.1016/j.asej.2022.101895).
- [5] A. Darzi, S. M. Gaweesh, M. M. Ahmed, and D. Novak, "Identifying the causes of drivers' hazardous states using driver characteristics, vehicle kinematics, and physiological measurements," *Frontiers Neurosci.*, vol. 12, p. 568, Aug. 2018, doi: [10.3389/fnins.2018.00568](https://doi.org/10.3389/fnins.2018.00568).
- [6] A. Altameem, A. Kumar, R. C. Poonia, S. Kumar, and A. K. J. Saudagar, "Early identification and detection of driver drowsiness by hybrid machine learning," *IEEE Access*, vol. 9, pp. 162805–162819, 2021, doi: [10.1109/ACCESS.2021.3131601](https://doi.org/10.1109/ACCESS.2021.3131601).
- [7] A. Kolus, "A systematic review on driver drowsiness detection using eye activity measures," *IEEE Access*, vol. 12, pp. 97969–97993, 2024, doi: [10.1109/ACCESS.2024.3424654](https://doi.org/10.1109/ACCESS.2024.3424654).
- [8] R. Hooda, V. Joshi, and M. Shah, "A comprehensive review of approaches to detect fatigue using machine learning techniques," *Chronic Diseases Transl. Med.*, vol. 8, no. 1, pp. 26–35, Mar. 2022, doi: [10.1016/j.cdtm.2021.07.002](https://doi.org/10.1016/j.cdtm.2021.07.002).
- [9] A. Kashevnik, R. Shchedrin, C. Kaiser, and A. Stocker, "Driver distraction detection methods: A literature review and framework," *IEEE Access*, vol. 9, pp. 60063–60076, 2021, doi: [10.1109/ACCESS.2021.3073599](https://doi.org/10.1109/ACCESS.2021.3073599).
- [10] B. Alshaqqa, A. S. Baquhaizel, M. E. A. Ouis, M. Boumehed, A. Ouamri, and M. Keche, "Driver drowsiness detection system," in *Proc. 8th Int. Workshop Syst., Signal Process. Appl. (WoSSPA)*, May 2013, pp. 151–155.
- [11] W. Tipprasert, T. Charoenpong, C. Chianrabutra, and C. Sukjamsri, "A method of driver's eyes closure and yawning detection for drowsiness analysis by infrared camera," in *Proc. 1st Int. Symp. Instrum., Control, Artif. Intell., Robot. (ICA-SYMP)*, Jan. 2019, pp. 61–64.
- [12] K. Rezaee, S. R. Alavi, M. Madanian, M. R. Ghezlbash, H. Khavari, and J. Haddadnia, "Real-time intelligent alarm system of driver fatigue based on video sequences," in *Proc. 1st RSII/ISM Int. Conf. Robot. Mechatronics (ICRoM)*, Tehran, Iran, Feb. 2013, pp. 378–383, doi: [10.1109/ICRoM.2013.6510137](https://doi.org/10.1109/ICRoM.2013.6510137).
- [13] C. J. de Naurois, C. Bourdin, A. Stratulat, E. Diaz, and J.-L. Vercher, "Detection and prediction of driver drowsiness using artificial neural network models," *Accident Anal. Prevention*, vol. 126, pp. 95–104, May 2019, doi: [10.1016/j.aap.2017.11.038](https://doi.org/10.1016/j.aap.2017.11.038).
- [14] A. Al Redhaei, Y. Albadawi, S. Mohamed, and A. Alnoman, "Realtime driver drowsiness detection using machine learning," in *Proc. Adv. Sci. Eng. Technol. Int. Conf. (ASET)*, Dubai, UAE, Feb. 2022, pp. 1–6, doi: [10.1109/ASET53988.2022.9734801](https://doi.org/10.1109/ASET53988.2022.9734801).
- [15] H. Lamaazi, A. Alqassab, R. A. Fadul, and R. Mizouni, "Smart edge-based driver drowsiness detection in mobile crowdsourcing," *IEEE Access*, vol. 11, pp. 21863–21872, 2023, doi: [10.1109/ACCESS.2023.3250834](https://doi.org/10.1109/ACCESS.2023.3250834).
- [16] Y. Du, P. Ma, X. Su, and Y. Zhang, "Driver fatigue detection based on eye state analysis," in *Proc. 11th Joint Conf. Inf. Sci.*, Apr. 2008, pp. 132–137.
- [17] I.-H. Choi, S. K. Hong, and Y.-G. Kim, "Real-time categorization of driver's gaze zone using the deep learning techniques," in *Proc. Int. Conf. Big Data Smart Comput. (BigComp)*, Hong Kong, Jan. 2016, pp. 143–148, doi: [10.1109/BIGCOMP.2016.7425813](https://doi.org/10.1109/BIGCOMP.2016.7425813).
- [18] M. Ramzan, A. Abid, M. Fayyaz, T. J. Alahmadi, H. Nobanee, and A. Rehman, "A novel hybrid approach for driver drowsiness detection using a custom deep learning model," *IEEE Access*, vol. 12, pp. 126866–126884, 2024, doi: [10.1109/ACCESS.2024.3438617](https://doi.org/10.1109/ACCESS.2024.3438617).
- [19] M. Venkateswarlu and V. R. R. Ch., "DrowsyDetectNet: Driver drowsiness detection using lightweight CNN with limited training data," *IEEE Access*, vol. 12, pp. 110476–110491, 2024, doi: [10.1109/ACCESS.2024.3440585](https://doi.org/10.1109/ACCESS.2024.3440585).
- [20] A. Tawari, K. H. Chen, and M. M. Trivedi, "Where is the driver looking: Analysis of head, eye and iris for robust gaze zone estimation," in *Proc. 17th Int. IEEE Conf. Intell. Transp. Syst. (ITSC)*, Qingdao, China, Oct. 2014, pp. 988–994, doi: [10.1109/ITSC.2014.6957817](https://doi.org/10.1109/ITSC.2014.6957817).
- [21] L. Zhang, H. Saito, L. Yang, and J. Wu, "Privacy-preserving federated transfer learning for driver drowsiness detection," *IEEE Access*, vol. 10, pp. 80565–80574, 2022, doi: [10.1109/ACCESS.2022.3192454](https://doi.org/10.1109/ACCESS.2022.3192454).
- [22] S. A. Dwivedi, A. Attry, and K. Singla, "Leveraging transfer learning for driver drowsiness detection," in *Advances in Data and Information Sciences*, vol. 318, S. Tiwari, M. C. Trivedi, M. L. Kolhe, K. Mishra, and B. K. Singh, Eds., Singapore: Springer, 2022, doi: [10.1007/978-981-16-5689-7_53](https://doi.org/10.1007/978-981-16-5689-7_53).
- [23] H. A. Madni, A. Raza, R. Sehar, N. Thalji, and L. Abualigah, "Novel transfer learning approach for driver drowsiness detection using eye movement behavior," *IEEE Access*, vol. 12, pp. 64765–64778, 2024, doi: [10.1109/ACCESS.2024.3392640](https://doi.org/10.1109/ACCESS.2024.3392640).
- [24] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.
- [25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [26] M. A. Assari and M. Rahmati, "Driver drowsiness detection using face expression recognition," in *Proc. IEEE Int. Conf. Signal Image Process. Appl. (ICSIPA)*, Kuala Lumpur, Malaysia, Nov. 2011, pp. 337–341, doi: [10.1109/ICSIPA.2011.6144162](https://doi.org/10.1109/ICSIPA.2011.6144162).
- [27] S. Sathasivam, A. K. Mahamad, S. Saon, A. Sidek, M. M. Som, and H. A. Ameen, "Drowsiness detection system using eye aspect ratio technique," in *Proc. IEEE Student Conf. Res. Develop. (SCOREd)*, Batu Pahat, Malaysia, Sep. 2020, pp. 448–452, doi: [10.1109/SCOREd50371.2020.9251035](https://doi.org/10.1109/SCOREd50371.2020.9251035).
- [28] B. Geetavani, N. V. K. Rao, N. Shalini, V. A. Reddy, A. Amrutha, and A. Sultana, "Drivers fatigue detection and alarming using ear and MOR values," in *Proc. 5th Int. Conf. I-SMAC (IoT Social, Mobile, Anal. Cloud) (I-SMAC)*, Palladam, India, Nov. 2021, pp. 1023–1029, doi: [10.1109/I-SMAC52330.2021.9640964](https://doi.org/10.1109/I-SMAC52330.2021.9640964).
- [29] Y. Jebraeli, Y. Sharafi, and M. Teshnehlab, "Driver drowsiness detection based on convolutional neural network architecture optimization using genetic algorithm," *IEEE Access*, vol. 12, pp. 45709–45726, 2024, doi: [10.1109/ACCESS.2024.3381999](https://doi.org/10.1109/ACCESS.2024.3381999).
- [30] R. K. Singh, A. Sarkar, R. K. Thakur, and C. S. Anoop, "A real-time heart-rate monitor using non-contact electrocardiogram for automotive drivers," in *Proc. IEEE 1st Int. Conf. Control, Meas. Instrum. (CMI)*, Kolkata, India, Jan. 2016, pp. 484–488, doi: [10.1109/CMI.2016.7413795](https://doi.org/10.1109/CMI.2016.7413795).
- [31] T. S. Mounika, P. H. Phanindra, N. S. Charan, Y. K. K. Reddy, and S. Govindu, "Driver drowsiness detection using eye aspect ratio (EAR), mouth aspect ratio (MAR), and driver distraction using head pose estimation," in *ICT Systems and Sustainability*, vol. 321, M. Tuba, S. Akashe, and A. Joshi, Eds., Singapore: Springer, doi: [10.1007/978-981-16-5987-4_63](https://doi.org/10.1007/978-981-16-5987-4_63).
- [32] A. Qudus, A. S. Zandi, L. Prest, and F. J. E. Comeau, "Using long short term memory and convolutional neural networks for driver drowsiness detection," *Accident Anal. Prevention*, vol. 156, Jun. 2021, Art. no. 106107, doi: [10.1016/j.aap.2021.106107](https://doi.org/10.1016/j.aap.2021.106107).
- [33] F. Murat, O. Yildirim, M. Talo, U. B. Baloglu, Y. Demir, and U. R. Acharya, "Application of deep learning techniques for heartbeats detection using ECG signals-analysis and review," *Comput. Biol. Med.*, vol. 120, May 2020, Art. no. 103726, doi: [10.1016/j.combiomed.2020.103726](https://doi.org/10.1016/j.combiomed.2020.103726).
- [34] A. Qayyum, F. Meriaudeau, and G. C. Y. Chan, "Classification of atrial fibrillation with pre-trained convolutional neural network models," in *Proc. IEEE-EMBS Conf. Biomed. Eng. Sci. (IECBES)*, Sarawak, Malaysia, Dec. 2018, pp. 594–599, doi: [10.1109/IECBES.2018.8626624](https://doi.org/10.1109/IECBES.2018.8626624).
- [35] P. N. Ranaware and R. A. Deshpande, "Detection of arrhythmia based on discrete wavelet transform using artificial neural network and support vector machine," in *Proc. Int. Conf. Commun. Signal Process. (ICCCSP)*, Melmaruvathur, India, Apr. 2016, pp. 1767–1770, doi: [10.1109/ICCCSP.2016.7754470](https://doi.org/10.1109/ICCCSP.2016.7754470).
- [36] E. Izci, M. A. Ozdemir, M. Degirmenci, and A. Akan, "Cardiac arrhythmia detection from 2D ECG images by using deep learning technique," in *Proc. Med. Technol. Congr. (TIPTeKNO)*, Izmir, Turkey, Oct. 2019, pp. 1–4, doi: [10.1109/TIPTeKNO.2019.8895011](https://doi.org/10.1109/TIPTeKNO.2019.8895011).
- [37] U. R. Acharya, H. Fujita, S. L. Oh, Y. Hagiwara, J. H. Tan, and M. Adam, "Application of deep convolutional neural network for automated detection of myocardial infarction using ECG signals," *Proc. Inf. Sci.*, vol. 415, pp. 190–198, Nov. 2017, doi: [10.1016/j.ins.2017.06.027](https://doi.org/10.1016/j.ins.2017.06.027).
- [38] B. K. Savas and Y. Becerikli, "Real time driver fatigue detection system based on multi-task ConNN," *IEEE Access*, vol. 8, pp. 12491–12498, 2020, doi: [10.1109/ACCESS.2020.2963960](https://doi.org/10.1109/ACCESS.2020.2963960).

- [39] V. Vijaypriya and M. Uma, "Facial feature-based drowsiness detection with multi-scale convolutional neural network," *IEEE Access*, vol. 11, pp. 63417–63429, 2023, doi: [10.1109/ACCESS.2023.3288008](https://doi.org/10.1109/ACCESS.2023.3288008).
- [40] X. Zhou, H. Ding, W. Wu, and Y. Zhang, "A real-time atrial fibrillation detection algorithm based on the instantaneous state of heart rate," *PLoS ONE*, vol. 10, no. 9, Sep. 2015, Art. no. e0136544, doi: [10.1371/journal.pone.0136544](https://doi.org/10.1371/journal.pone.0136544).
- [41] Z. Yao, Z. Zhu, and Y. Chen, "Atrial fibrillation detection by multi-scale convolutional neural networks," in *Proc. 20th Int. Conf. Inf. Fusion (Fusion)*, China, Jul. 2017, pp. 1–6, doi: [10.23919/ICIF.2017.8009782](https://doi.org/10.23919/ICIF.2017.8009782).
- [42] Y. Albadawi, M. Takruri, and M. Awad, "A review of recent developments in driver drowsiness detection systems," *Sensors*, vol. 22, no. 5, p. 2069, Mar. 2022, doi: [10.3390/s22052069](https://doi.org/10.3390/s22052069).
- [43] A. Gupta, A. Tiwari, A. S. Chauhan, and S. Gupta, "A comprehensive review of driver drowsiness detection systems and applied technologies," in *Proc. IEEE Int. Conf. Comput., Power Commun. Technol. (IC2PCT)*, Greater Noida, India, Feb. 2024, pp. 1624–1629, doi: [10.1109/IC2PCT60090.2024.10486282](https://doi.org/10.1109/IC2PCT60090.2024.10486282).
- [44] I. Jahan, K. M. A. Uddin, S. A. Murad, M. S. U. Miah, T. Z. Khan, M. Masud, S. Aljahdali, and A. K. Bairagi, "4D: A real-time driver drowsiness detector using deep learning," *Electronics*, vol. 12, no. 1, p. 235, Jan. 2023, doi: [10.3390/electronics12010235](https://doi.org/10.3390/electronics12010235).
- [45] C. H. Praharsa and A. Poulse, "CBAM VGG16: An efficient driver distraction classification using CBAM embedded VGG16 architecture," *Comput. Biol. Med.*, vol. 180, Sep. 2024, Art. no. 108945, doi: [10.1016/j.combiomed.2024.108945](https://doi.org/10.1016/j.combiomed.2024.108945).
- [46] K. V. C. Golosinda, J. Van D. Marcellones, L. J. P. Ampongol, N. P. Magloyuan, and P. D. Cerna, "DriSafePh: An IoT based realtime driver drowsiness detection system using hybrid machine learning algorithm," in *Proc. 14th Int. Conf. Softw. Technol. Eng. (ICSTE)*, Aug. 2024, pp. 258–265, doi: [10.1109/icste63875.2024.00051](https://doi.org/10.1109/icste63875.2024.00051).
- [47] K. Xu, J. Yin, J. Kan, F. Dong, and K. Chen, "Vision-based driver drowsiness detection system," in *Proc. 2nd Int. Conf. Mach. Learn., Cloud Comput. Intell. Mining (MLCCIM)*, Jul. 2023, pp. 442–447.
- [48] F. Wei, J. Yang, Y. Wang, L. Lin, and H. Zhang, "Prior knowledge-guided multi-information graph convolutional network for driver drowsiness detection," *Expert Syst. Appl.*, vol. 275, May 2025, Art. no. 127028, doi: [10.1016/j.eswa.2025.127028](https://doi.org/10.1016/j.eswa.2025.127028).
- [49] S. Priyanka, S. Shanthi, A. S. Kumar, and V. Praveen, "Data fusion for driver drowsiness recognition: A multimodal perspective," *Egyptian Informat. J.*, vol. 27, Sep. 2024, Art. no. 100529, doi: [10.1016/j.eij.2024.100529](https://doi.org/10.1016/j.eij.2024.100529).



SINDHU VIDYATHAN DIXITH (Member, IEEE) received the bachelor's degree in electrical and electronics engineering from the National Institute of Engineering, Mysuru, India, in 2017, and the master's degree in electrical engineering from San José State University, in 2022. She is currently a System Engineer with Qualcomm, Santa Clara, CA, USA. Her work is focused on the functional safety domain of automotive. Before her graduate studies, she was associated with

Bosch as an Embedded Software Engineer for passive safety systems, automotive-airbags, from 2017 to 2020. She received a certificate for completing a project on the control and monitoring of loads for microgrids in remote areas having no electricity and the Center of Renewable Energy and Sustainable Technologies, Mysuru. She has also received certification in Model-based Systems Engineering. Her research interests include embedded systems, deep learning, and computer architecture. This includes combining deep learning techniques and algorithms and applying them to embedded systems, applying these models to a set of drowsiness-related datasets, performing an experimental analysis, and comparing accuracies to obtain efficient models.



SHRIKANT JADHAV (Senior Member, IEEE) received the Ph.D. degree in electrical engineering from North Carolina A&T State University, Greensboro, in 2018. He joined SJSU, as an Assistant Professor of electrical engineering, in the Fall of 2021. Before joining SJSU, he was an Assistant Professor for a year at Fort Lewis College, Durango, CO, USA. He was hired as a Postdoctoral Researcher to work on the "A Mobile High-Performance Computing System for Deployment in the Field" project supported by the DoD grant, from 2018 to 2020. During the Ph.D. and Postdoctoral Research period, he mentored several high school, undergraduate, and graduate students. His research interests include the development of methodologies for accelerating computationally intensive applications, such as image processing, bioinformatics, and machine learning. He has also served as a technical committee member and the session chair at several conferences and journals.



YOUNGSOO KIM received the B.S. and M.S. degrees in electronic engineering from Inha University, South Korea, and the Ph.D. degree in electrical engineering from North Carolina State University, Raleigh. After the Ph.D. degree, he received postdoctoral training at North Carolina State University, working on a high-performance computing (HPC) project. Since 2023, he has been with the Electrical Engineering Department, University of Minnesota Duluth, USA, as an Assistant Professor. His research interests include embedded systems, development of parallel algorithms, and parallel computational structures on field-programmable devices, including the IoT devices.



NAVEENKUMAR JAYAKUMAR received the Ph.D. degree in computer engineering and possessing expertise in storage systems, cybersecurity, and intelligent computing. He is an Associate Professor at the Vellore Institute of Technology, Vellore. He has over a decade of academic experience and has collaborated with industry in various capacities at CDAC, EMC Corporation, QED Baton, and L&T Infotech, among others. His scholarship spans Scopus-indexed journals, proceedings, and book chapters, as well as a co-authored national-level textbook, with research supported by grants and patents in the fields of smart computational systems and security. A recognized Ph.D. guide, he has supervised numerous dissertations, led curriculum design and NBA accreditation efforts, and delivered expert lectures across academia and industry. His current interests focus on computational storage, cybersecurity for cyber-physical systems, edge computing, and applied machine learning.

...